Delco

Manual técnico do projeto

Motor de orquestração de workflows com Python, Django, MySQL, Redis e WebSockets



Copyright (©) 2026 Erik Pimenta

Versão 1.2.0 · Novembro de 2026 · Licença MIT

Sumário

1. Visão geral 3
Novidades da versão 1.2.0 3
Tecnologias 4
2. Como rodar 6
Contas de demonstração 6
3. Escrevendo workflows 8
Parâmetros tipados 9
O objeto ctx, skip, fail e retry_on 10
Testar sem servidor 10
4. Guia das telas 11
Workflows, versões e código 12
Execuções e tarefas 17
Sistema 23
Minha conta, preferências e paletas 23
5. Arquitetura 26
6. Integrações 28
7. Linha de comando 30
8. API 31
9. Configuração 33
10. Operação em produção 35
11. Segurança 36
12. Testes e qualidade 37
13. Solução de problemas 38

Capítulo 1

Visão geral

O Delco é um motor de orquestração de workflows de código aberto, no estilo do Apache Airflow e do Temporal. Você descreve um pipeline em Python como um grafo de tarefas com dependências; o Delco agenda as execuções, distribui as tarefas entre workers, repete o que falhar com espera exponencial e mostra tudo em tempo real num painel web.

O diferencial de engenharia é a confiabilidade: o estado de cada execução vive no MySQL. Se o servidor cair no passo 23 de um fluxo de 50, ao voltar o Delco sabe exatamente onde parou e refaz só o passo 23.

Para que serve

Qualquer rotina com várias etapas que precisa rodar sozinha, no horário certo, sem perder o fio quando algo dá errado: ETL e relatórios, integrações com fornecedores instáveis, sincronizações, backups, migrações em lotes, processamento de arquivos e notificações.

O que o Delco faz

Para quem escreve workflows

- Workflows como código Python, com dependências, agenda cron, fila, tempo limite e retentativas por tarefa.
- Parâmetros tipados com `Param`: o painel monta o formulário e todos os caminhos validam do mesmo jeito.
- `ctx.skip()`, `ctx.fail()` e `retry_on` para separar erros passageiros de definitivos.
- Tempo limite da execução inteira com `run_timeout`.
- `delco new` e `delco dev run` para criar e testar um workflow sem subir nada.

Para quem opera

- Painel com grafo ao vivo, grade, linha do tempo, logs que chegam enquanto a tarefa roda e quadro de vagas dos workers.
- Disparo com formulário, reprocessamento de períodos, retomada da falha, marcação manual de tarefas e ações em lote.
- Pausa de agendas e de workers, tela Sistema, `delco doctor`, métricas para o Prometheus e healthchecks.
- Avisos no Slack, Discord, Teams, Google Chat, Telegram, por e-mail e por webhook assinado.

Para quem administra

- Três papéis (Leitura, Operador e Administrador), tokens pessoais com validade e auditoria com filtros e exportação.
- Configuração conferida ao subir, limites de chamadas e limpeza automática do histórico.

Novidades da versão 1.2.0

Novidade	Onde ler
Parâmetros tipados com tipo, padrão, opções e limites; formulário no painel e erro por campo na API.	Capítulo 3

Novidade	Onde ler
Skip, Fail e retry_on: tarefa pulada pela própria lógica, falha definitiva e retentativas só para erros passageiros.	Capítulo 3
Tempo limite da execução (`run_timeout`), com o motivo gravado e aviso às integrações.	Capítulos 3 e 5
delco new e delco dev run; `wf.run_local()` para testes.	Capítulos 3 e 7
Versões e código de cada workflow no painel, com o que mudou entre versões.	Capítulo 4
Reprocessar período (backfill) com prévia, pelo painel, API e CLI.	Capítulos 4, 7 e 8
Marcar tarefa como sucesso, falha ou pulada e seguir a execução.	Capítulo 4
Ações em lote em execuções e agendas; **pausar worker**.	Capítulo 4
Tela Sistema, **delco doctor**, sinal do agendador, **métricas**, **healthcheck** e logs em JSON.	Capítulos 4 e 10
Telegram, e-mail, evento **voltou a funcionar**, gatilho assinado, reenvio de entregas e limite de chamadas.	Capítulo 6
Preferências da conta: tema, cinco paletas de cores, fuso, densidade, itens por página, avisos do navegador.	Capítulo 4
Auditoria com filtros e CSV; observação editável; exclusão de execuções.	Capítulos 4 e 11
Banco: colunas `params_schema`, `source_code`, `error`, `preferences` e `config`; origem `backfill`; estados `paused` e `skipped`.	Manual do banco de dados

Tecnologias

Camada	Tecnologia	Papel
Linguagem	Python 3.13	SDK, agendador, workers, API e CLI
Framework	Django 6.1	Modelos, migrações, comandos de gerenciamento, configuração e segurança
API	Django REST Framework + drf-spectacular	REST com permissões por papel e documentação OpenAPI/Swagger
Servidor	Daphne (ASGI)	HTTP e WebSocket no mesmo processo
Banco	MySQL 8.4	Fonte da verdade de todo o estado
Fila	Redis 7 Streams	Entrega das tarefas com grupo de consumidores; sinais e limites de chamadas
Tempo real	Django Channels + Redis pub/sub	Eventos de execução, tarefa, worker e log pelo WebSocket
Notificações	httpx e smtplib	Webhooks, Telegram e e-mail com tempo limite e assinatura HMAC
Painel	HTML, CSS e JavaScript (sem build)	Servido pelo próprio Django com WhiteNoise
Agenda	croniter	Cron com fuso e próximos horários
Qualidade	Pytest, pytest-cov e Ruff	103 testes com MySQL e Redis reais e lint no CI

Camada	Tecnologia	Papel
Execução	Docker Compose	MySQL, Redis, API, agendador e workers com healthcheck

Capítulo 2

Como rodar

Requisitos

- Docker com Docker Compose (Docker Desktop no Windows ou no macOS).
- Cerca de 1,5 GB livres para as imagens e a porta 8080 livre (troque com `DELCO_PORT`).

Primeira execução

1. Baixe o repositório (**Code → Download ZIP** no GitHub ou `git clone`).
2. **Windows:** dois cliques em `iniciar.cmd`. **Linux ou macOS:** `./iniciar.sh` (ou `make subir`).
3. Na primeira vez o script cria o `.env` com senha do MySQL, chave secreta e token de métricas aleatórios.
4. O Docker sobe MySQL, Redis, API, agendador e dois workers, e o painel abre em **http://localhost:8080**.
5. Confira tudo com `docker compose exec api delco doctor`.

```
cp .env.example .env             # troque DELCO_DB_PASSWORD e DELCO_SECRET_KEY
docker compose up -d --build
docker compose up -d --scale worker=4
docker compose exec api delco doctor
```

Comandos do dia a dia

Ação	Comando
Subir ou atualizar	`docker compose up -d --build` · `make subir`
Acompanhar logs	`docker compose logs -f api scheduler worker` · `make logs`
Mais workers	`docker compose up -d --scale worker=4`
Diagnóstico	`docker compose exec api delco doctor` · `make doutor`
Novo workflow	`delco new nome` · `make novo NOME=nome`
Parar	`docker compose down` (os dados ficam nos volumes)
Começar do zero	`docker compose down -v`

Contas de demonstração

Todas usam a senha **Delco@2026** e existem só para testes. Os nomes são fictícios e os e-mails usam o domínio de exemplo `delco.dev`.

Papel	E-mail	Nome
Administrador	`admin@delco.dev`	Helena Prado
Operador	`operador@delco.dev`	Tiago Moura
Leitura	`leitura@delco.dev`	Bruna Sales

> Para usar de verdade, suba com `DELCO_SEED=false` num banco vazio e crie o administrador com `docker compose exec api delco users create --role admin --email voce@empresa.com --name "Seu Nome"`. A tela Sistema e o `delco doctor` avisam se alguma conta de demonstração continuar ativa com a senha pública.

Rodar sem Docker

```
pip install -r requirements.txt && pip install -e .
export DELCO_DATABASE_URL=mysql://delco:senha@127.0.0.1:3306/delco
export DELCO_REDIS_URL=redis://127.0.0.1:6379/0
delco api --port 8080 &     # aplica as migrações ao subir
delco scheduler &
delco worker --concurrency 4
```

Capítulo 3

Escrevendo workflows

Estrutura de um arquivo

Cada arquivo `.py` da pasta `workflows/` (ou de `DELCO_WORKFLOWS_DIR`) define um ou mais objetos `Workflow`. O agendador relê a pasta a cada 30 segundos; arquivos começando com `_` são ignorados. O jeito mais rápido de começar é `delco new nome --schedule "0 7 * * *"`, que cria o arquivo já com parâmetros e duas tarefas.

```
from delco import Fail, Param, Workflow

vendas = Workflow(
    "relatorio_vendas",
    description="Extrai, agrega e publica o relatório do dia.",
    schedule="0 7 * * *",
    timezone="America/Sao_Paulo",
    params={
        "dias": Param(1, min=1, max=30, description="Quantos dias processar"),
        "formato": Param("csv", choices=["csv", "xlsx"]),
    },
    run_timeout=1800,
)

@vendas.task(retries=3, timeout=120, retry_on=(ConnectionError, TimeoutError))
def extrair(ctx):
    linhas = buscar_pedidos(ctx.params["dias"])
    if not linhas:
        ctx.skip("Nenhum pedido no período")
    return {"linhas": len(linhas)}

@vendas.task(depends_on=[extrair])
def publicar(ctx):
    if ctx.upstream["extrair"]["linhas"] > 1_000_000:
        raise Fail("Volume fora do normal")
    ctx.log("publicado")
```

Opções do workflow

Parâmetro	Padrão	Descrição
`name`	obrigatório	Letras minúsculas, números e `_` (até 63)
`description`	vazio	Texto mostrado no painel
`schedule`	`None`	Cron de 5 campos ou `@daily`, `@hourly`…; `None` = só manual
`timezone`	`America/Sao_Paulo`	Fuso da agenda
`params`	`{}`	Valores simples ou `Param(...)`
`max_active_runs`	`1`	Execuções simultâneas; as outras esperam na fila

Parâmetro	Padrão	Descrição
`run_timeout`	`None`	Segundos até a execução inteira ser encerrada como falha
`tags`	`()`	Etiquetas para busca
`default_retries` · `default_retry_delay` · `default_timeout` · `default_queue`	`2` · `5` · `300` · `default`	Padrões das tarefas

Opções da tarefa

Opção	Padrão	Descrição
`depends_on`	`[]`	Tarefas (função ou nome) que precisam terminar antes
`retries`	`2`	Novas tentativas depois da primeira falha
`retry_delay` · `backoff` · `max_retry_delay`	`5` · `2` · `600`	Espera = `retry_delay × backoff^(n-1)` com ±10%, até o teto
`retry_on`	qualquer erro	Tupla de exceções que geram nova tentativa; as outras falham de vez
`timeout`	`300`	Segundos por tentativa; `None` = sem limite
`queue`	`default`	Fila; os workers escolhem as filas que consomem
`trigger_rule`	`all_success`	`all_done` roda sempre; `one_failed` só se algo falhar
`description`	docstring	Texto da tarefa no painel

Para gerar tarefas em laço, use `wf.add_task(nome, funcao, depends_on=[...])`. Funções `async def` também funcionam.

Parâmetros tipados

`Param` declara o que cada parâmetro aceita. A mesma regra vale no formulário do painel, na API, na CLI, no gatilho por URL, no reprocessamento e no `delco dev run`. Erros voltam por campo: `{"params.dias": "Precisa ser no máximo 30."}`.

Argumento	Descrição
primeiro argumento	Valor padrão; sem ele, o parâmetro é obrigatório
`type`	`str`, `int`, `float`, `bool`, `date`, `list` ou `dict` (deduzido do padrão)
`choices`	Lista de opções (vira um seletor no formulário)
`min` · `max`	Limites de números ou tamanho de textos e listas
`description` · `label`	Ajuda e rótulo do campo
`required`	Força obrigatório mesmo com padrão
`secret`	Mascara o valor no painel, na API e na auditoria

Textos vindos da CLI e de formulários são convertidos (`"3"` vira `3`, `"sim"` vira `True`). Datas chegam à tarefa como `"AAAA-MM-DD"`. Parâmetros declarados sem `Param` continuam livres, como nas versões anteriores. Um workflow agendado não pode ter parâmetro obrigatório sem padrão.

O objeto ctx, skip, fail e retry_on

Atributo ou método	O que faz
`ctx.params`	Parâmetros já validados
`ctx.upstream`	Resultado de cada dependência, por nome
`ctx.log(msg, level)` · `ctx.warning` · `ctx.error`	Linhas de log; `print` e o `logging` também viram log
`ctx.progress(50, "metade")`	Barra de andamento no painel
`ctx.skip("motivo")` ou `raise Skip(...)`	A tarefa termina como **pulada**; dependentes com `all_success` também são puladas e a execução conta como sucesso
`ctx.fail("motivo")` ou `raise Fail(...)`	Falha definitiva, sem retentativas (`retry=True` para repetir)
`ctx.idempotency_key`	Igual em todas as tentativas: use para não repetir efeitos externos
`ctx.logical_date` · `ctx.attempt` · `ctx.is_retry` · `ctx.is_last_attempt`	Horário agendado (ou do reprocessamento) e tentativa atual

> Com `retry_on=(ConnectionError, TimeoutError)`, uma queda de rede é repetida, mas um `KeyError` por erro de código falha na hora, sem esperar as retentativas. O log da tentativa avisa: “Erro marcado como definitivo”.

Testar sem servidor

```
delco dev run relatorio_vendas -p dias=3          # roda aqui, na ordem, com logs
delco dev run relatorio_vendas --no-retries --output
delco dev run relatorio_vendas --json            # para scripts
delco validate workflows                         # confere os arquivos
```

O `delco dev run` não precisa de MySQL, Redis nem workers: executa as tarefas no próprio processo, respeitando dependências, regras de disparo, `skip`, `Fail` e retentativas (sem a espera). Dentro dos seus testes, `vendas.run_local({"dias": 3})` devolve o estado, as tentativas, os logs e o resultado de cada tarefa.

Versões

O Delco calcula uma impressão digital do grafo (tarefas, dependências e opções). Quando ela muda, nasce uma versão nova; cada execução guarda a versão com que começou. O código do arquivo também fica guardado e aparece na aba **Código**. Mudar só o corpo de uma função não cria versão.

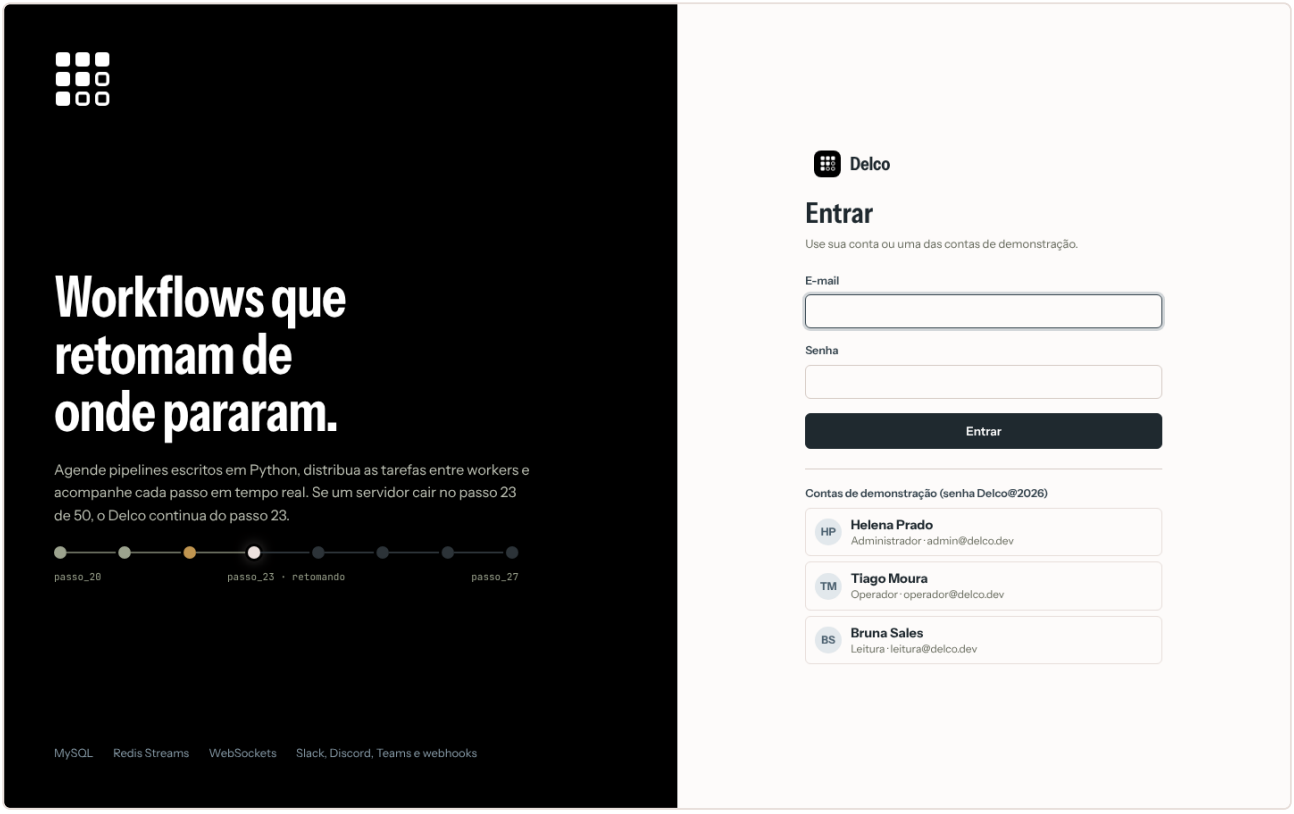
Exemplos prontos: `ola_delco`, `relatorio_vendas_diario` (leque paralelo, parâmetros, `skip`, `Fail` e `retry_on`), `sincronizar_catalogo`, `backup_noturno` (fila `critical` e `run_timeout`) e `migracao_em_50_passos`.

Capítulo 4

Guia das telas

O painel tem uma barra lateral com Visão geral, Workflows, Execuções, Workers, Desempenho, Integrações, Sistema e a documentação da API. `Ctrl K` (ou `⌘K`) abre a paleta de comandos e `?` lista os atalhos. Se nenhum agendador estiver ativo, um aviso aparece no topo de todas as telas.

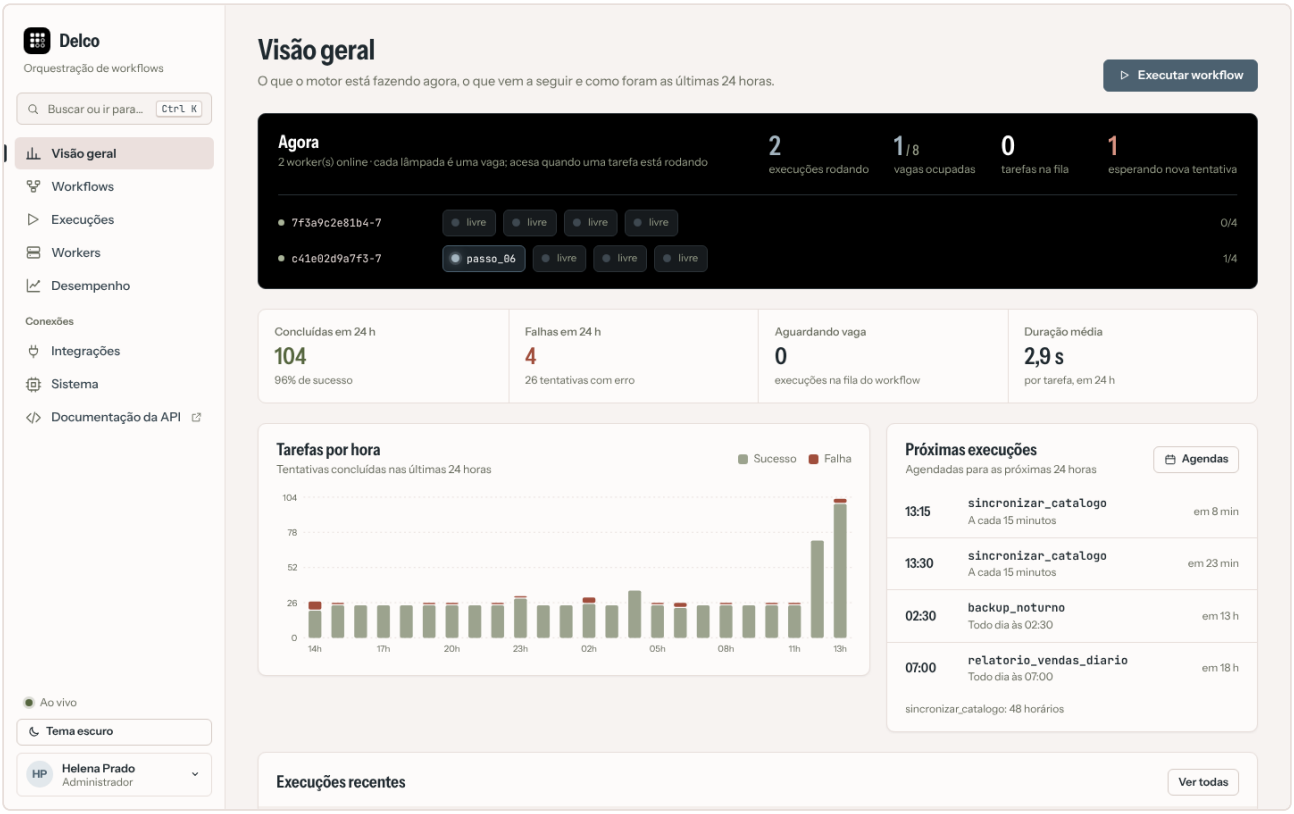
Entrada



Entrada com as contas de demonstração

Visão geral

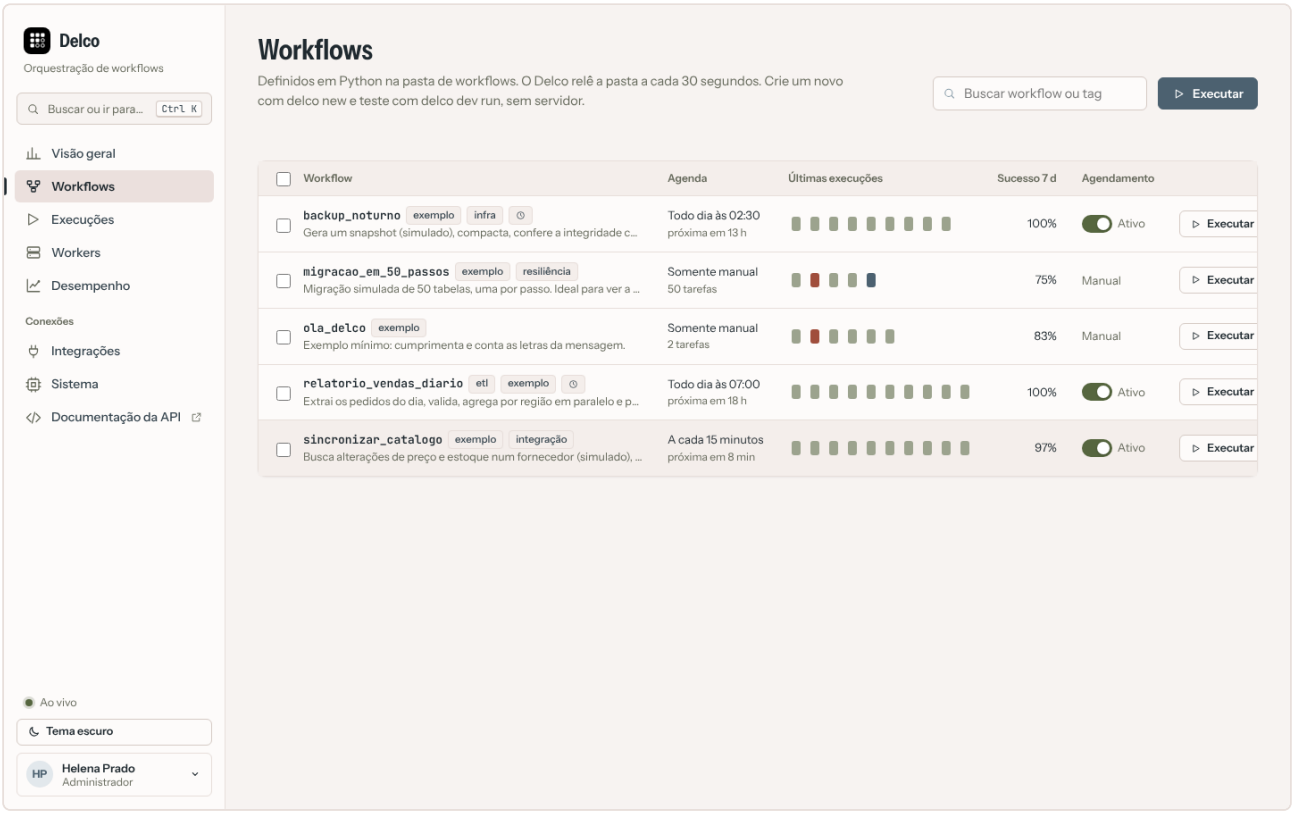
O quadro **Agora** mostra cada worker e suas vagas, acesas quando uma tarefa está rodando (workers pausados aparecem com as vagas tracejadas). Abaixo, indicadores de 24 horas, tarefas por hora, próximas execuções e as execuções recentes.



Visão geral

Workflows

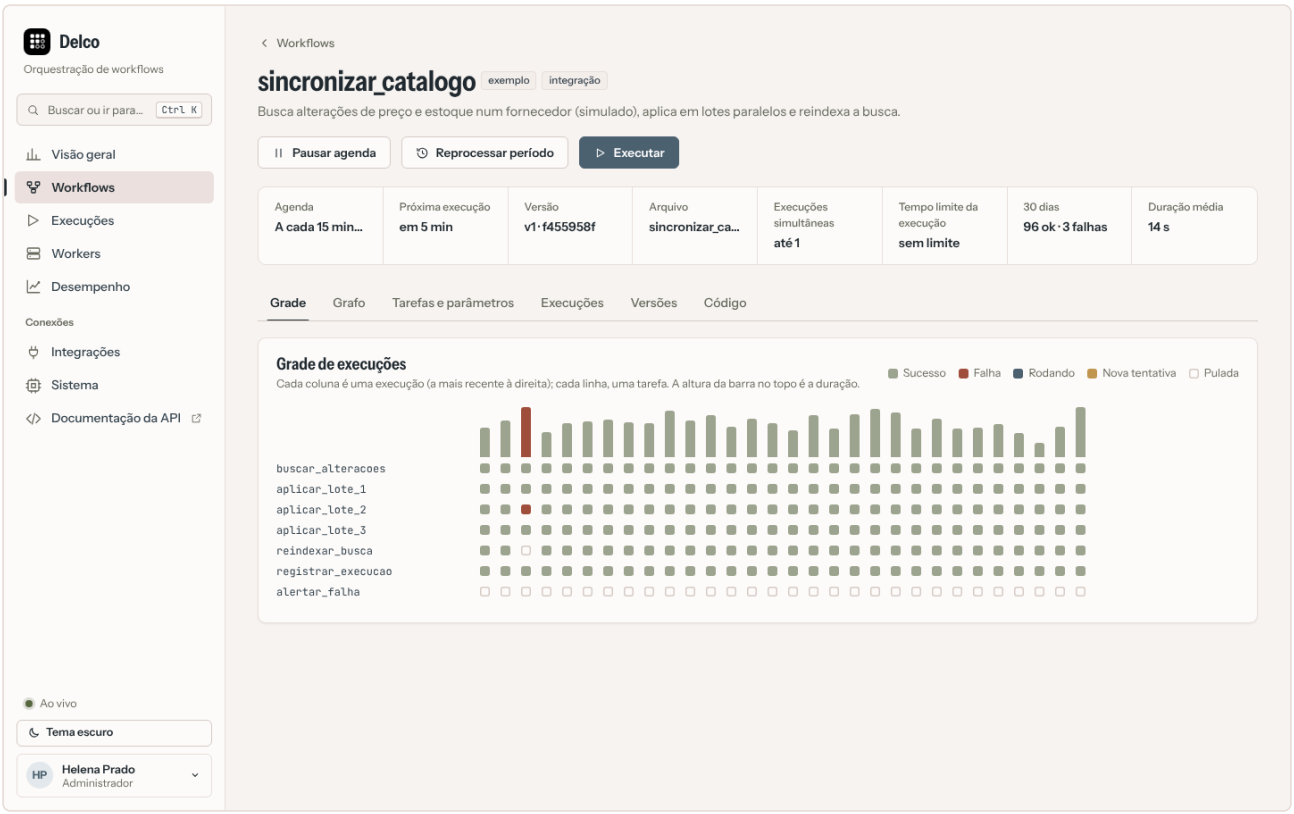
A lista mostra agenda, últimas execuções e taxa de sucesso. Marque vários para **pausar ou retomar as agendas** de uma vez.



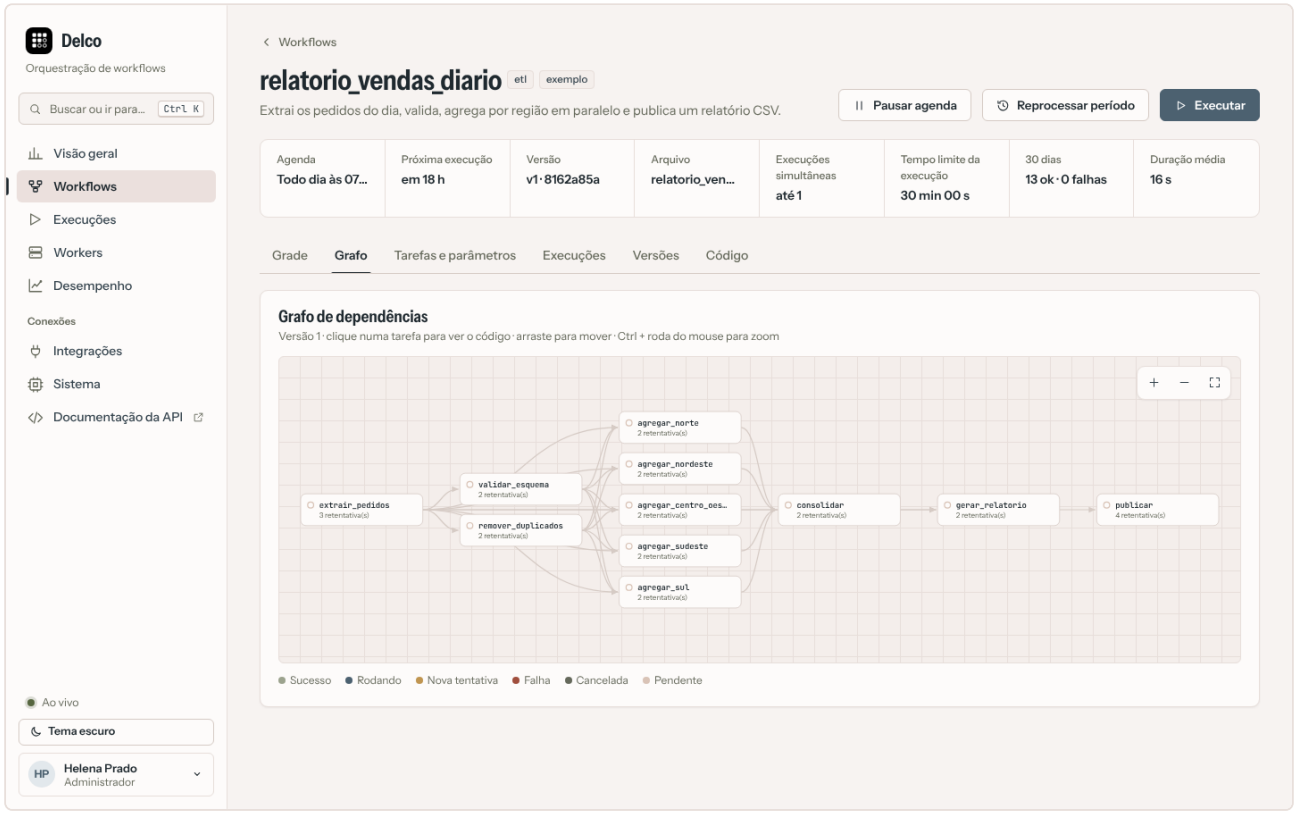
Lista de workflows com seleção em lote

Detalhe do workflow

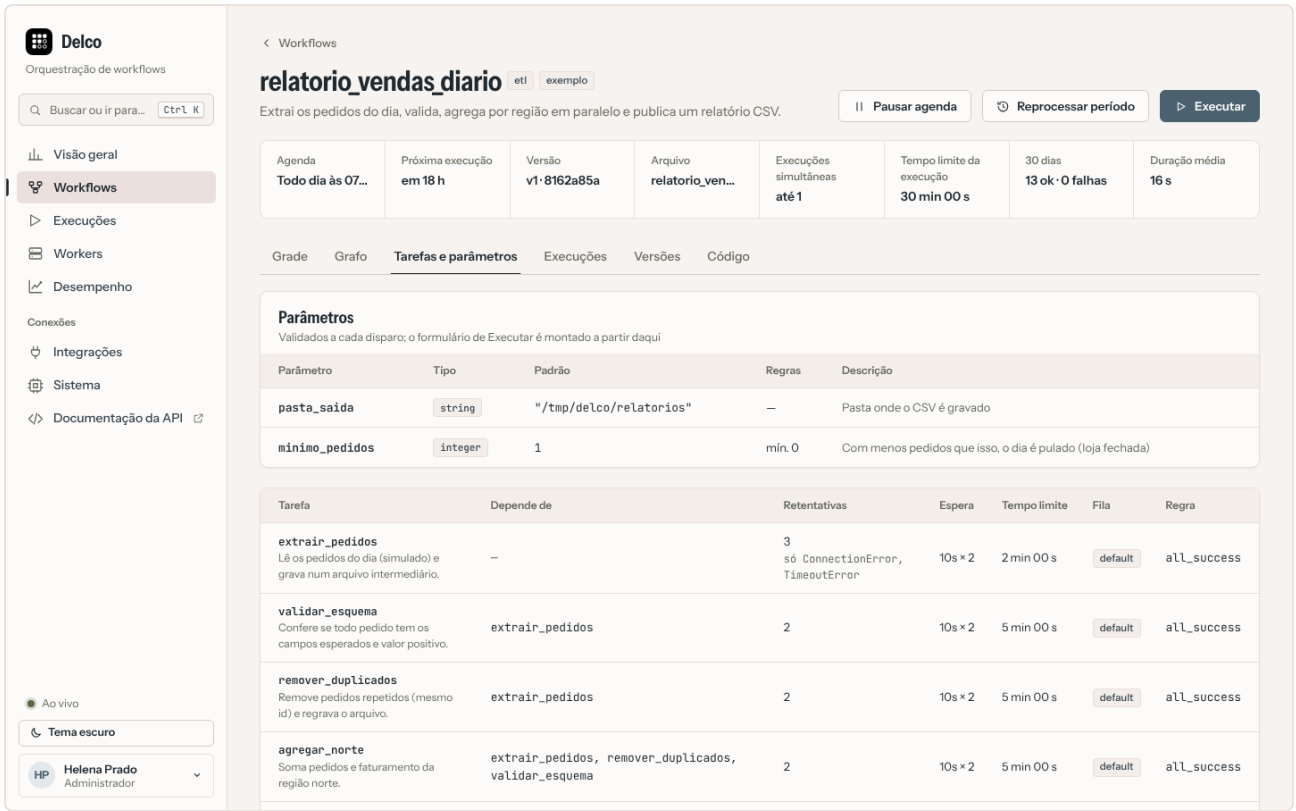
Fatos (agenda, próxima execução, versão, arquivo, limites) e as abas **Grade**, **Grafo**, **Tarefas e parâmetros**, **Execuções**, **Versões** e **Código**. A aba aberta fica no endereço da página, para compartilhar o link.



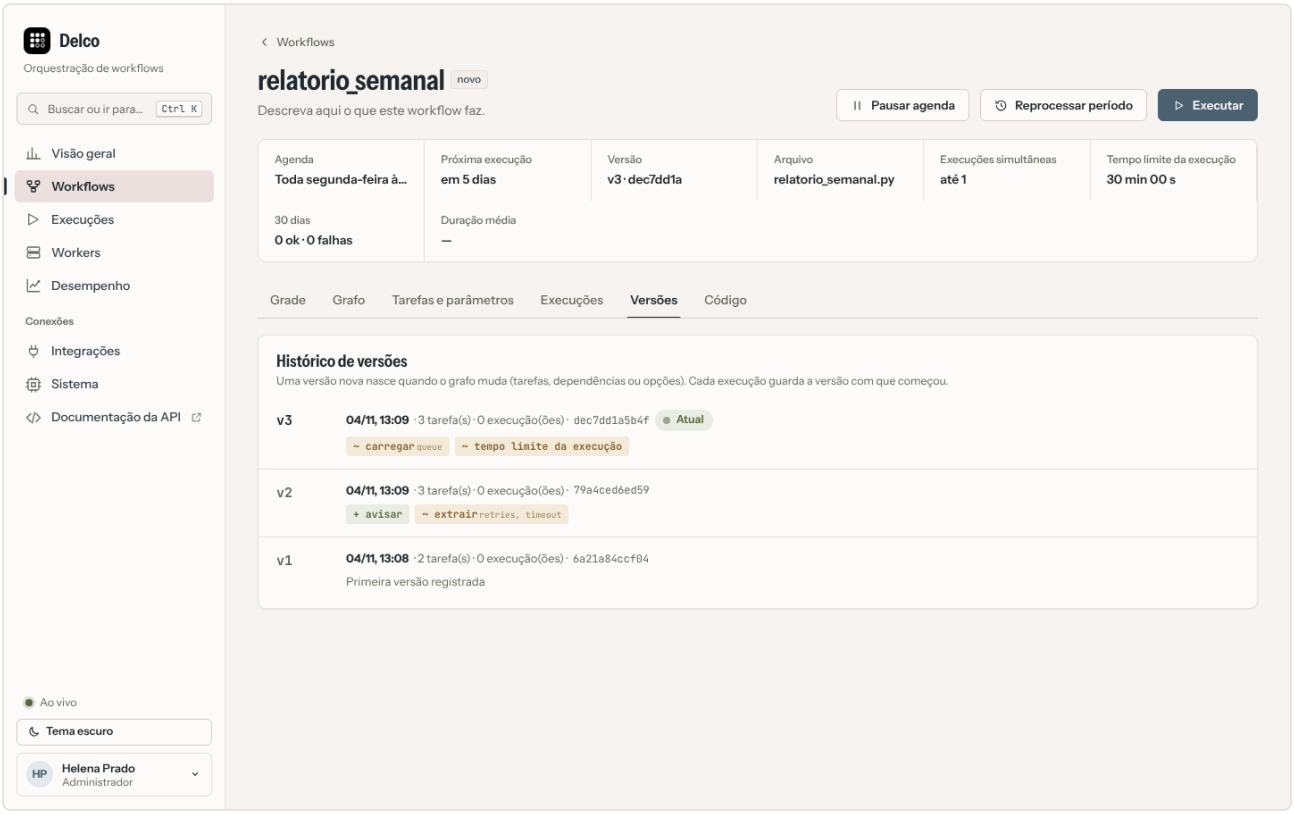
Grade: cada coluna é uma execução e cada linha, uma tarefa



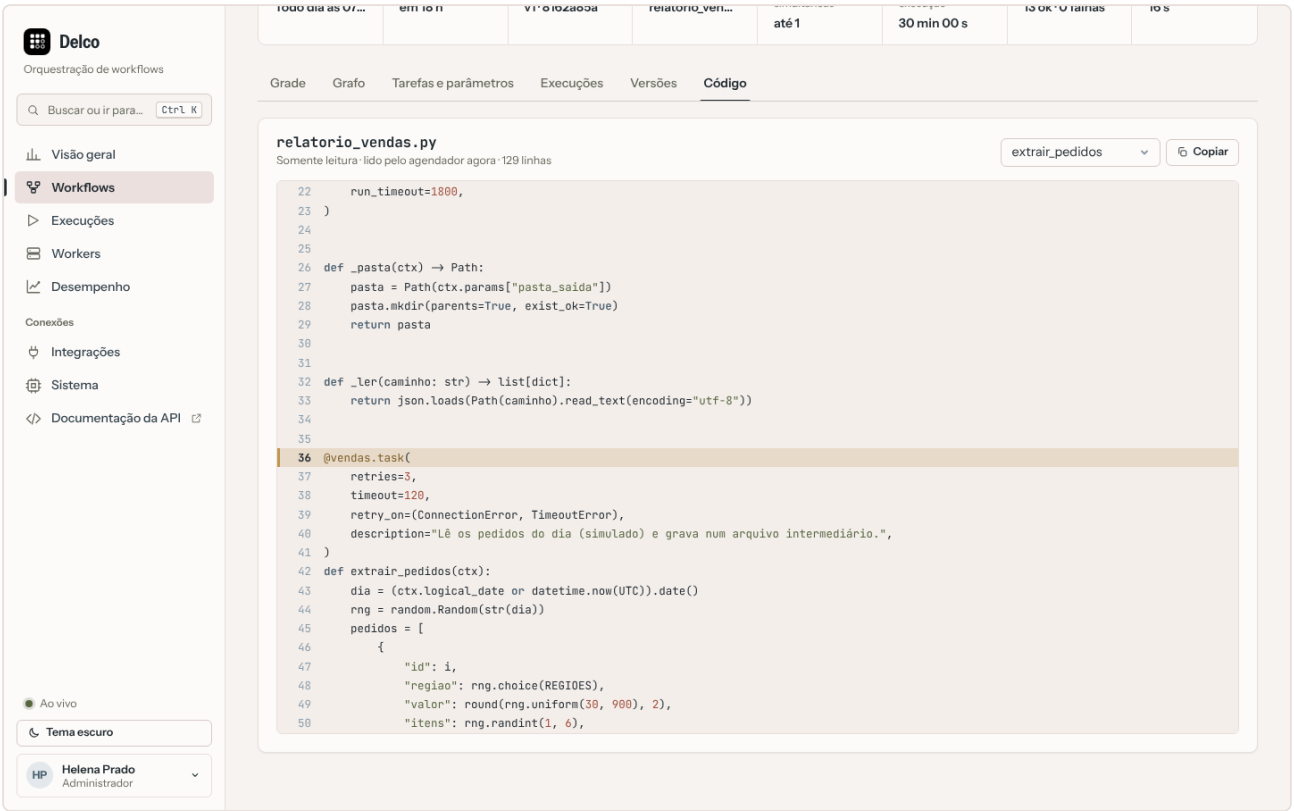
Grafo de dependências; clicar numa tarefa abre o código dela



Tarefas e parâmetros: tipos, padrões, regras e retentativas



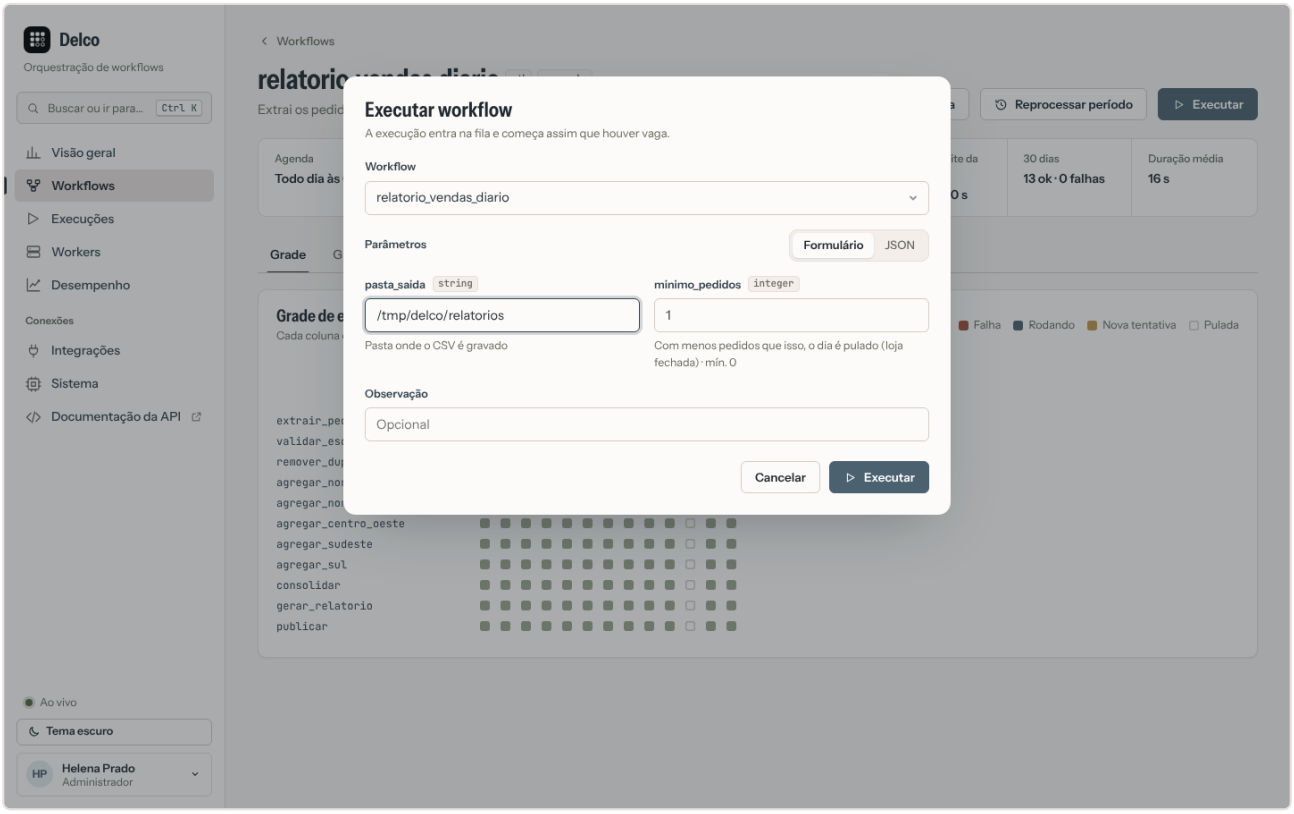
Versões: o que entrou (+), saiu (−) e mudou (~) em cada versão



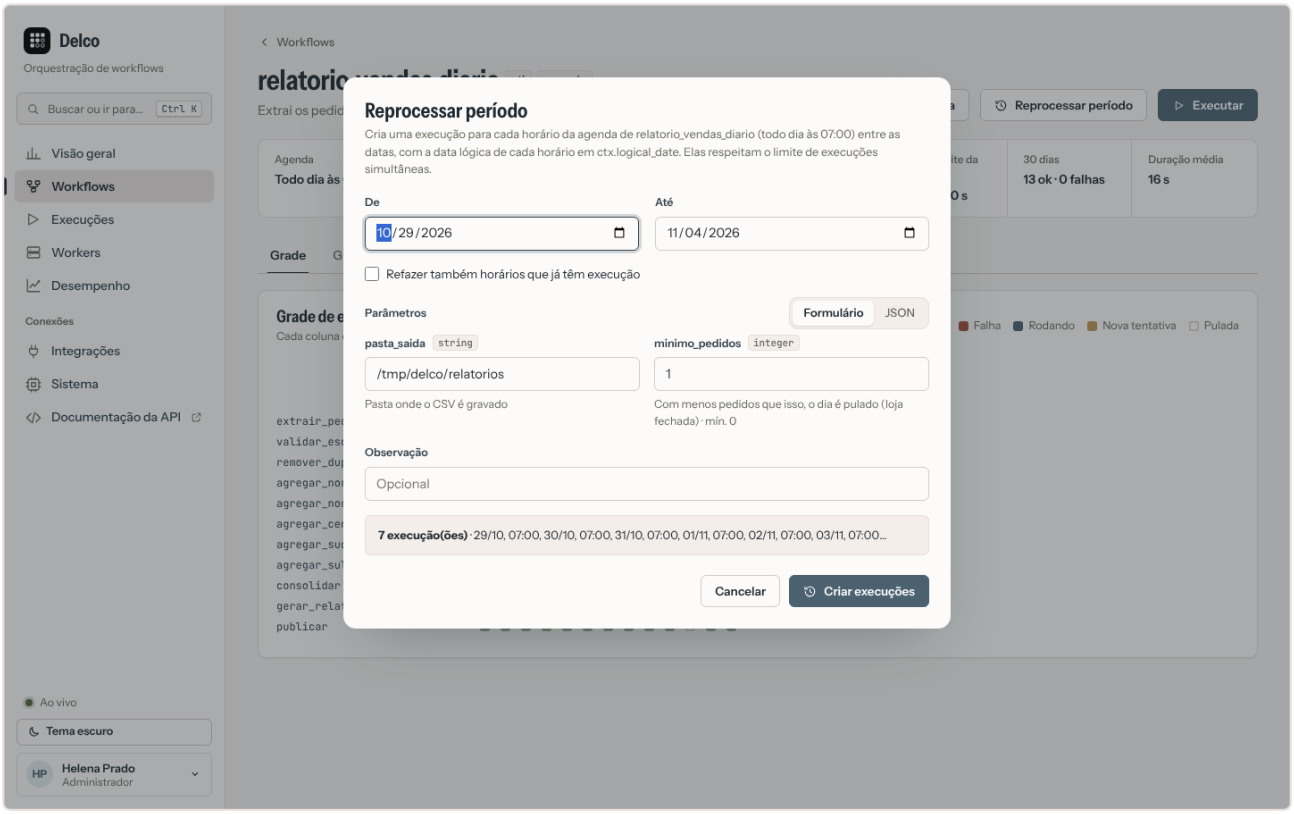
Código do arquivo com destaque de sintaxe e salto para cada tarefa

Executar e reprocessar

Executar abre um formulário montado a partir dos parâmetros (ou um editor JSON, para quem prefere). Erros aparecem embaixo de cada campo. **Reprocessar período** (workflows com agenda) mostra quantos horários existem entre as datas, quais já têm execução e cria uma execução por horário, com a data lógica certa.



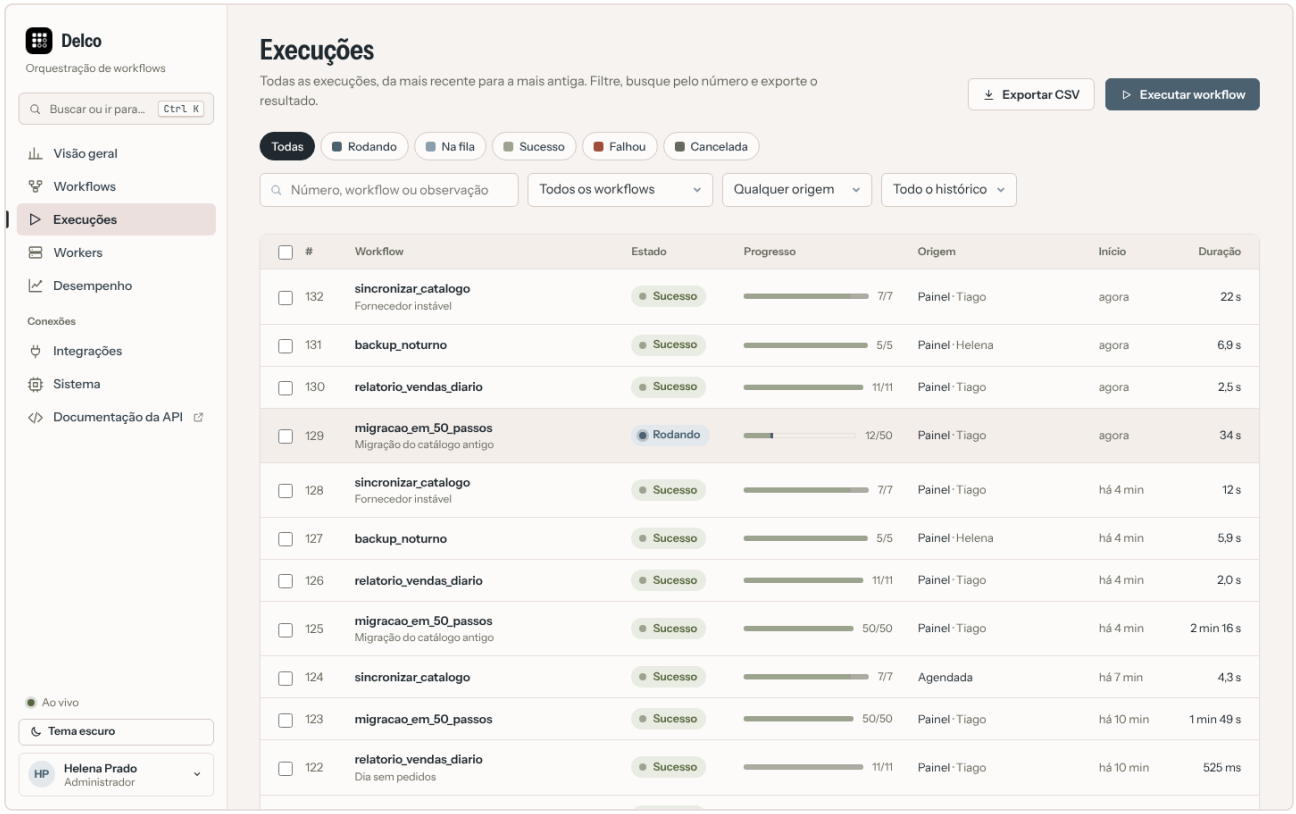
Executar workflow com formulário de parâmetros



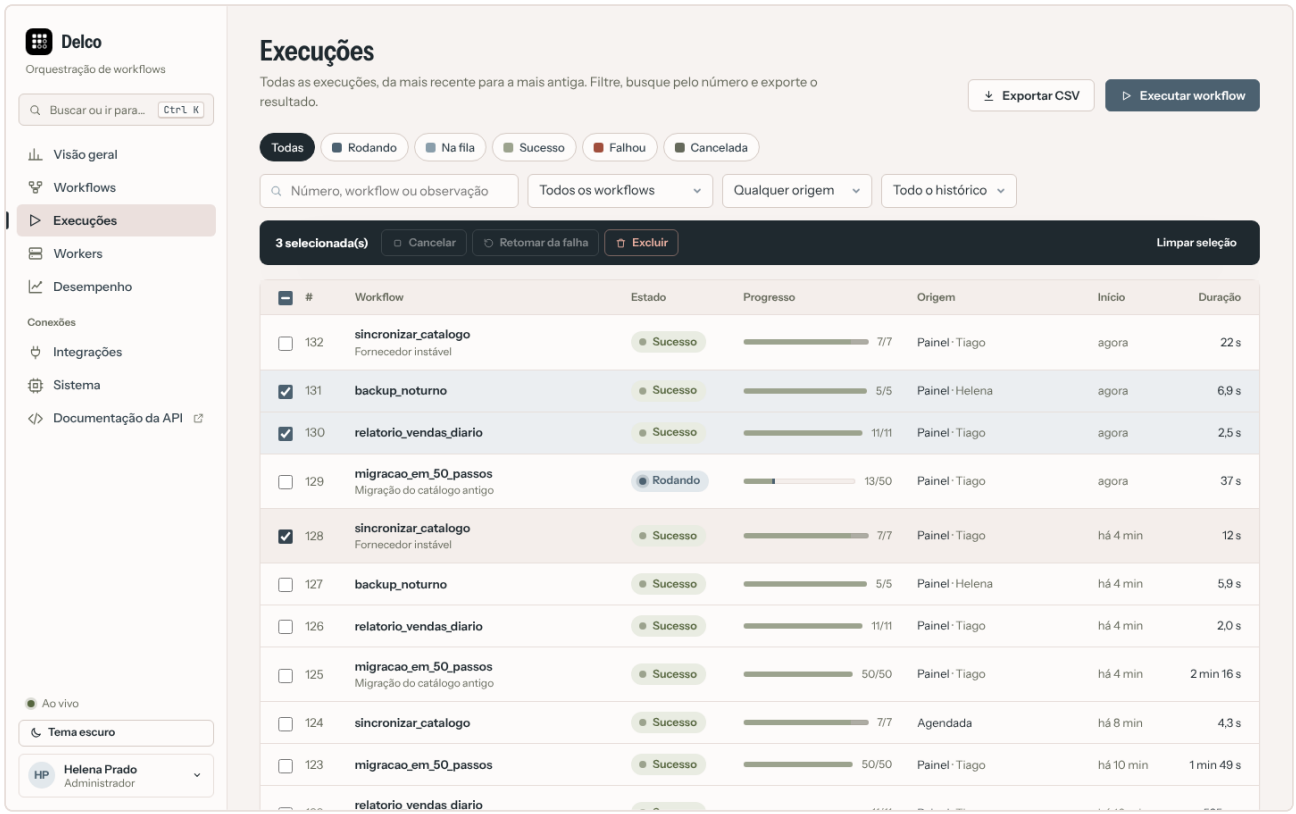
Reprocessar período com prévia

Execuções

Filtros por estado, workflow, origem (inclusive **Reprocessamento**) e período, busca por número ou observação e **Exportar CSV**. Marque execuções para **cancelar**, **retomar da falha** ou **excluir** (administrador) em lote. O tamanho da página segue a preferência da conta.



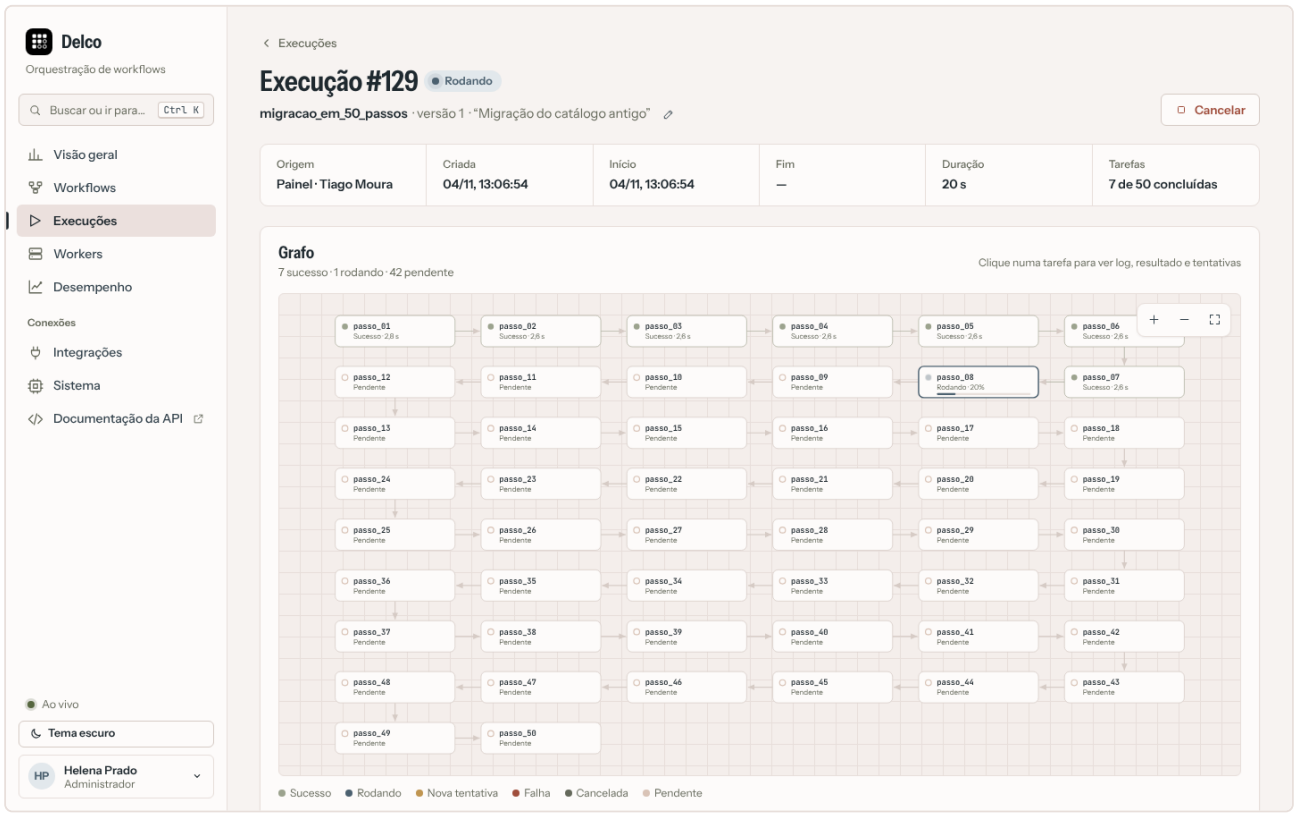
Execuções com filtros



Barra de ações em lote

Execução

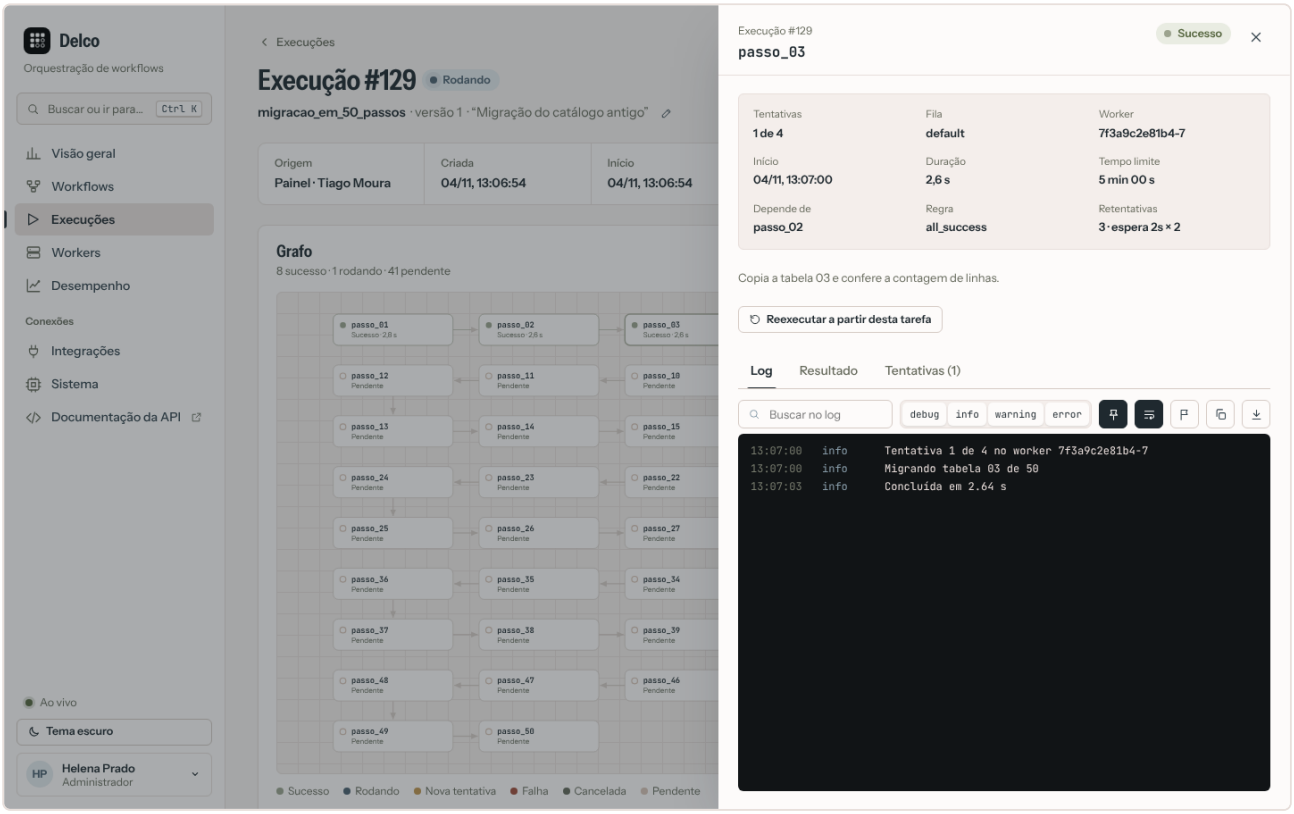
Grafo ao vivo, tarefas, linha do tempo e parâmetros. O motivo da falha da execução (como o tempo limite) aparece em destaque e a duração vem comparada com a mediana das execuções anteriores (“12% mais lenta”). A observação pode ser editada pelo lápis, e o administrador pode excluir uma execução terminada.



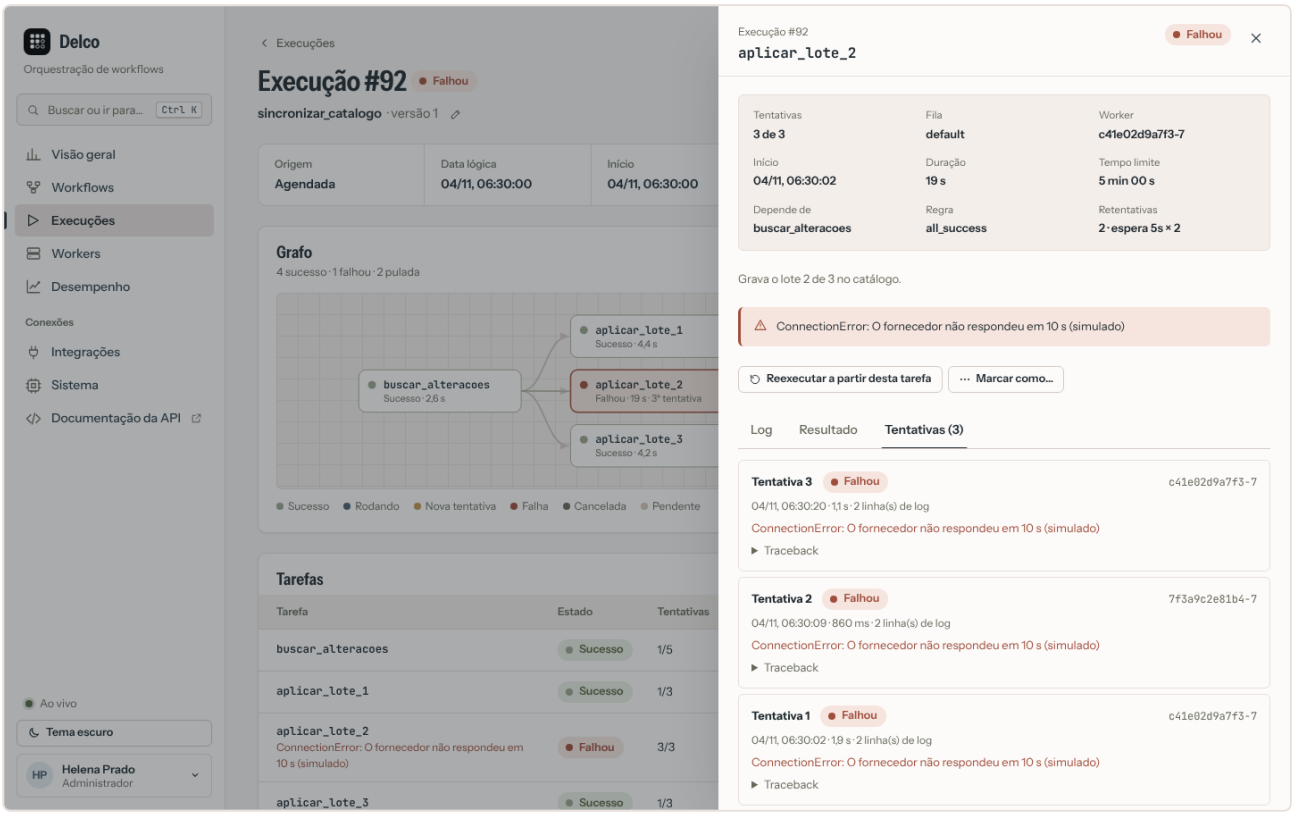
Execução de 50 passos em andamento

Gaveta da tarefa

Log ao vivo com busca, filtro por nível, **acompanhar o fim**, **quebrar linhas**, **ir para o primeiro erro**, copiar e baixar. Abas de resultado e tentativas, com traceback. Em execuções terminadas, o operador pode **reexecutar a partir desta tarefa** ou **marcar como sucesso, falha ou pulada**: marcar como sucesso devolve à fila as tarefas que tinham sido puladas por causa dela, e a execução continua.



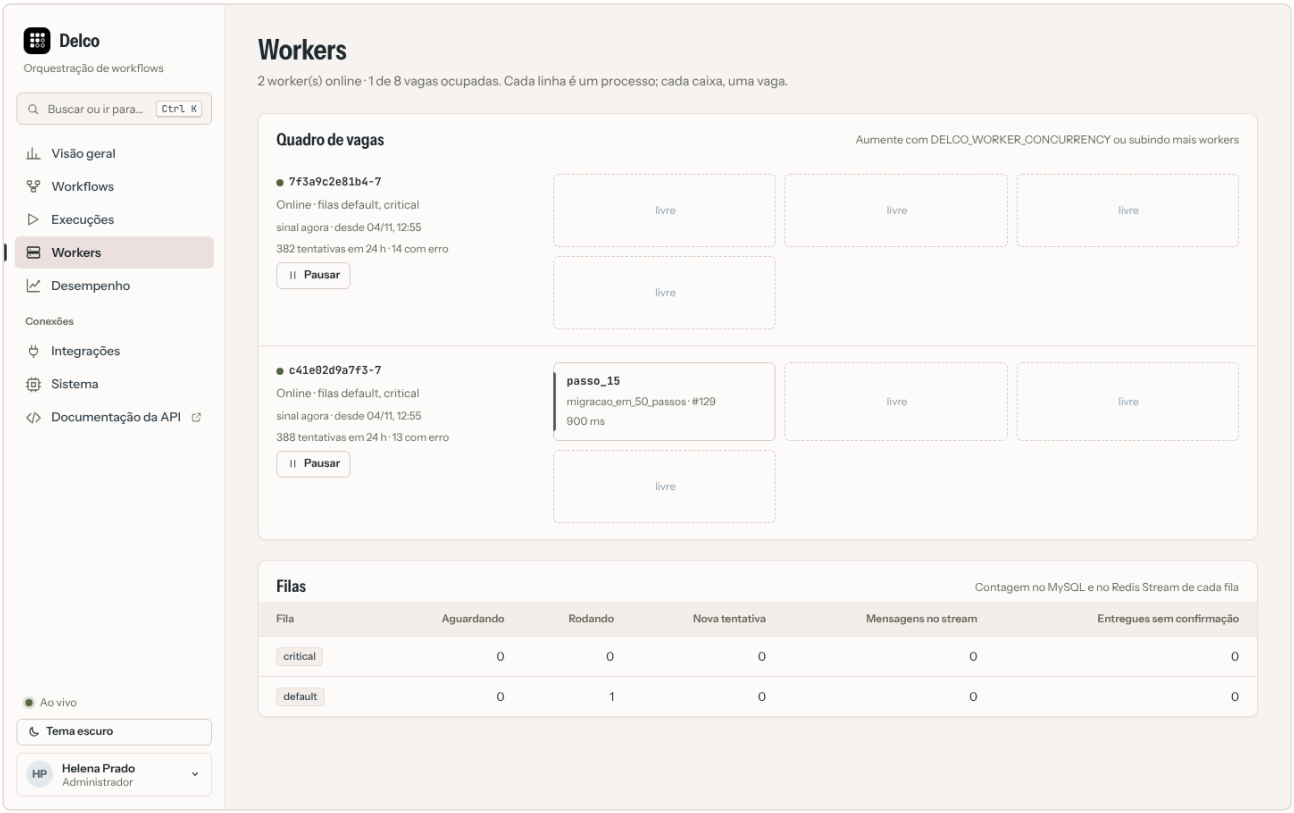
Gaveta da tarefa



Tentativas de uma tarefa que falhou

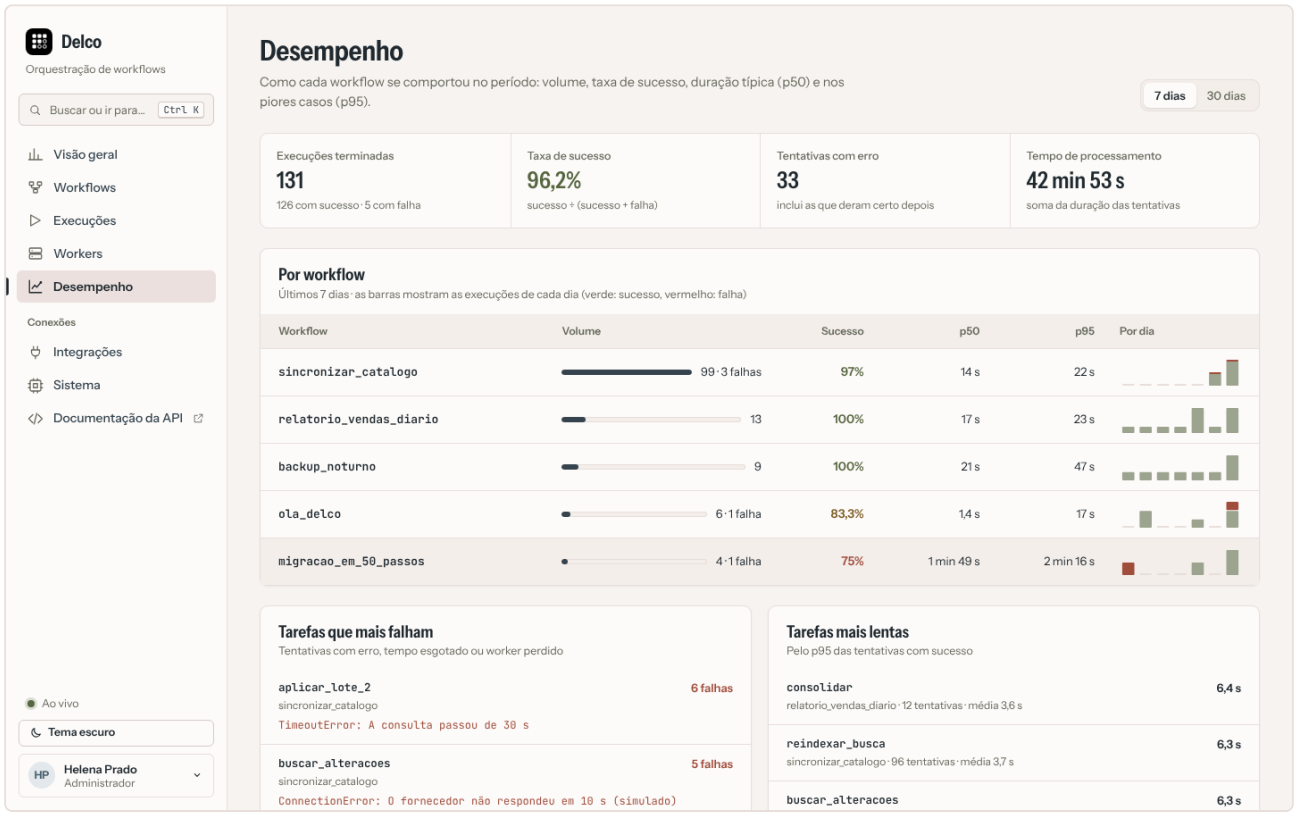
Workers

Cada worker com suas vagas e o que está rodando, e as filas no MySQL e no Redis. **Pausar** faz o worker terminar o que está rodando e parar de pegar tarefas novas; **Retomar** o devolve ao trabalho.



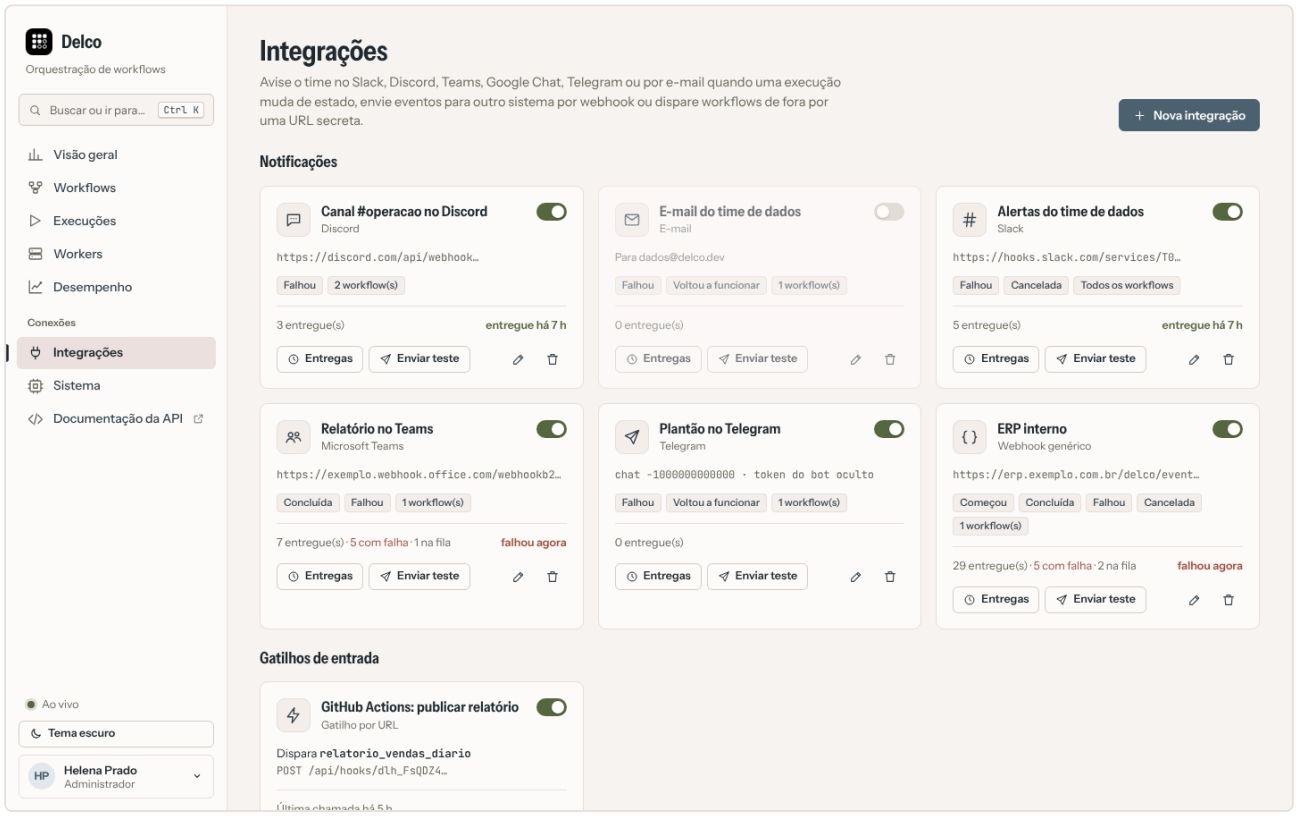
Quadro de vagas e filas

Desempenho



Taxa de sucesso, p50 e p95 por workflow, tarefas que mais falham e mais lentas

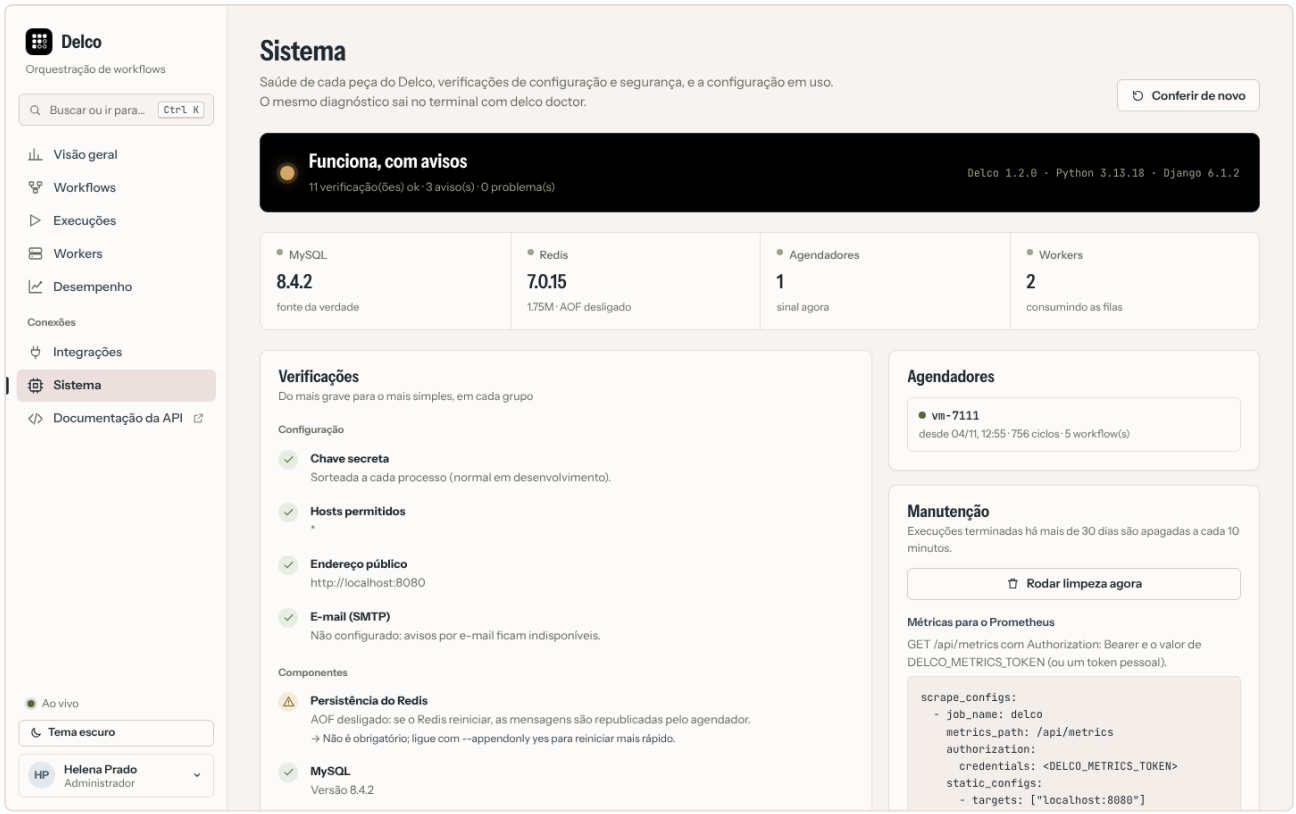
Integrações



Notificações e gatilhos de entrada

Sistema

Só para administradores. Resume a saúde (tudo certo, avisos ou problemas), mostra MySQL, Redis, agendadores e workers, lista as verificações com o que fazer em cada aviso, os agendadores com seu último sinal e erro, a configuração em uso (senhas mascaradas), o exemplo de configuração do Prometheus e o botão **Rodar limpeza agora**.



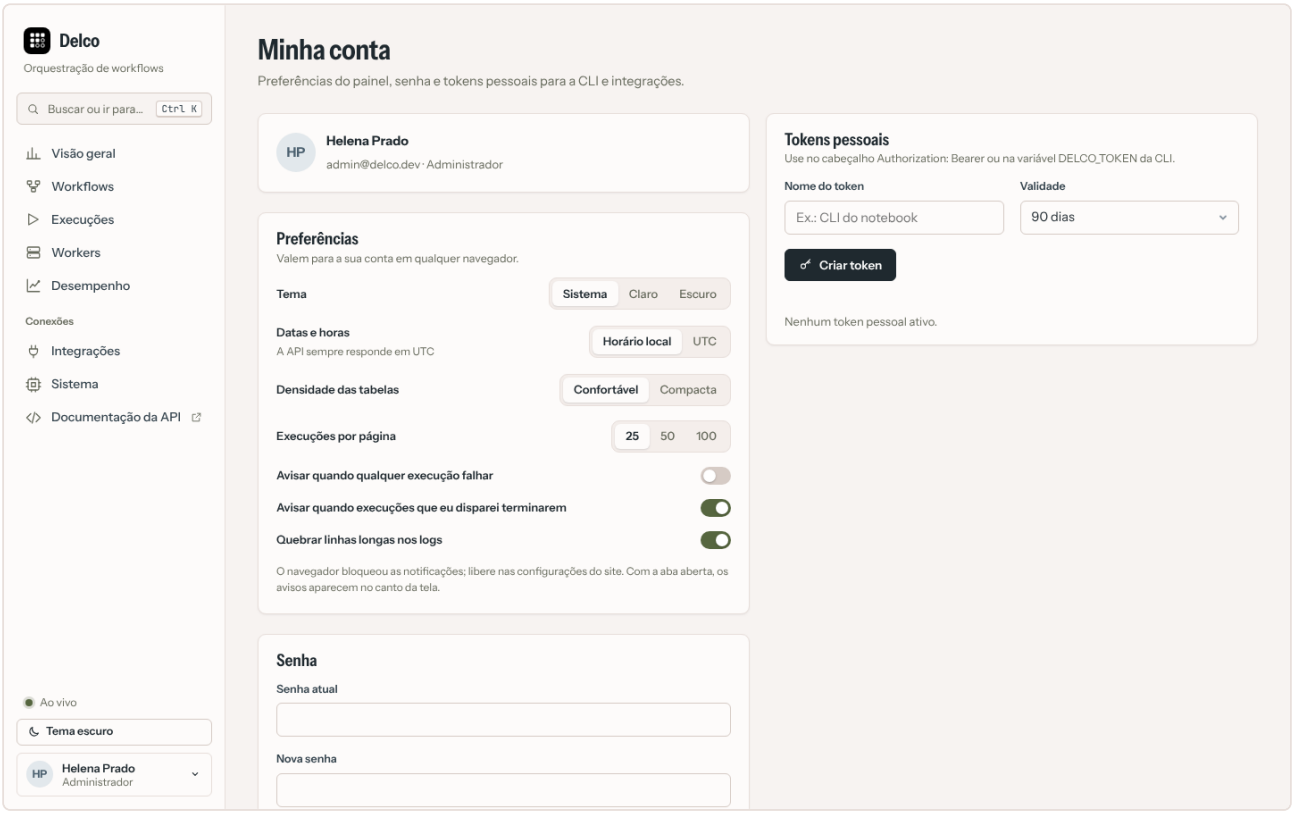
Tela Sistema

Minha conta, preferências e paletas

As preferências ficam na conta e valem em qualquer navegador: tema (sistema, claro ou escuro), **paleta de cores**, datas no horário local ou em UTC, densidade das tabelas, execuções por página, avisos do navegador (falhas e execuções que você disparou) e quebra de linha nos logs. A paleta tem cinco opções, escolhidas pelas amostras de cor:



Cada paleta tem versão clara e escura; a marca preta e branca do Delco não muda. A primeira é a paleta original do Delco.



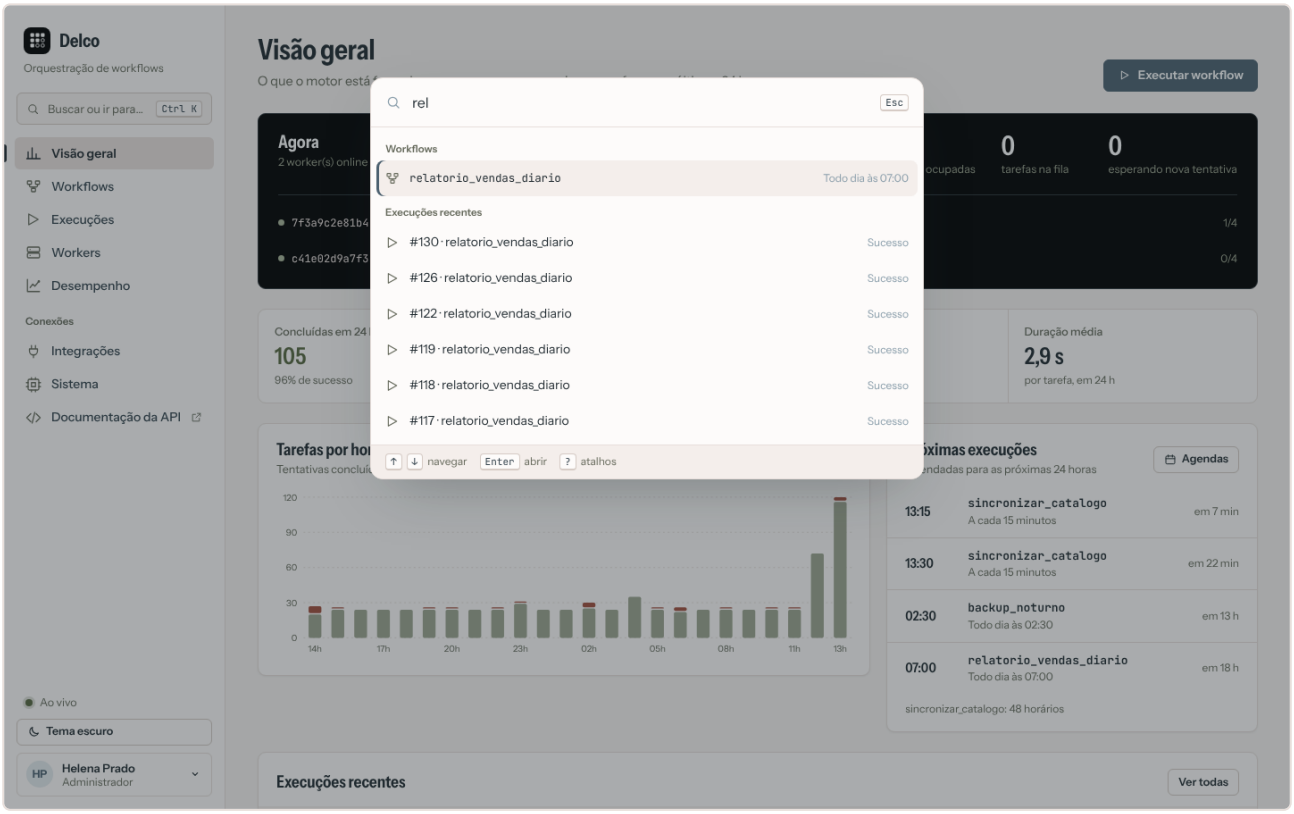
Minha conta: preferências, senha e tokens pessoais

Usuários e auditoria

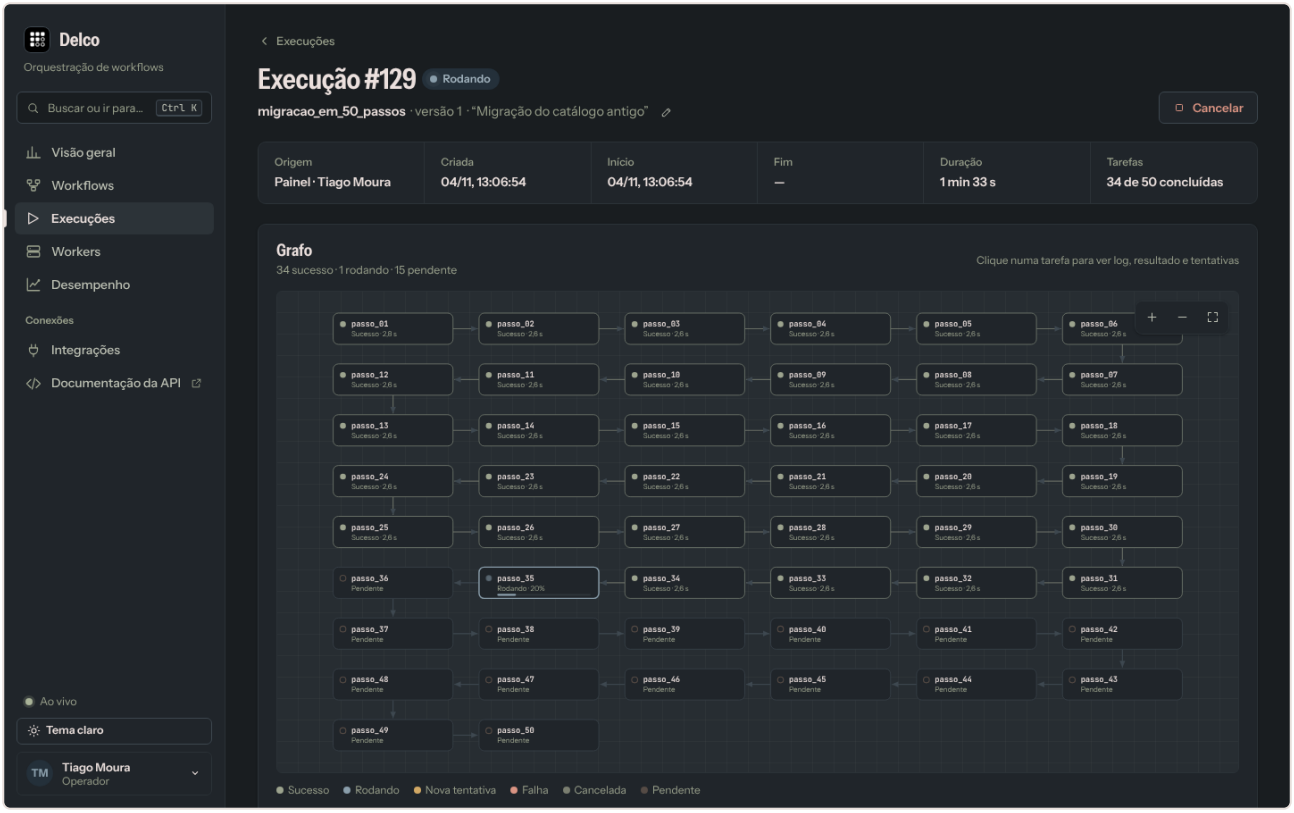
Administradores criam e editam usuários e consultam a auditoria com filtros por ação, pessoa, período e busca, com paginação e exportação em CSV.

Paleta de comandos e atalhos

`Ctrl K` busca telas, workflows e execuções (`#118`) e executa ações: executar workflow, trocar tema, trocar a paleta de cores, preferências, pausar ou retomar todas as agendas. Atalhos: `g v`, `g w`, `g e`, `g k`, `g d`, `g i`, `g c`, `g u`, `g s`, `/` para buscar e `x` para executar.



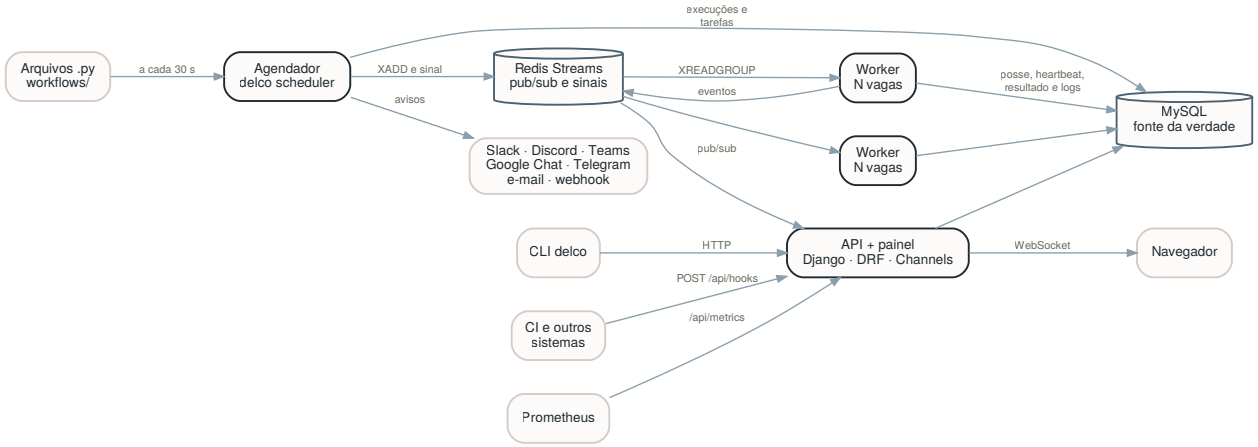
Paleta de comandos



Tema escuro

Capítulo 5

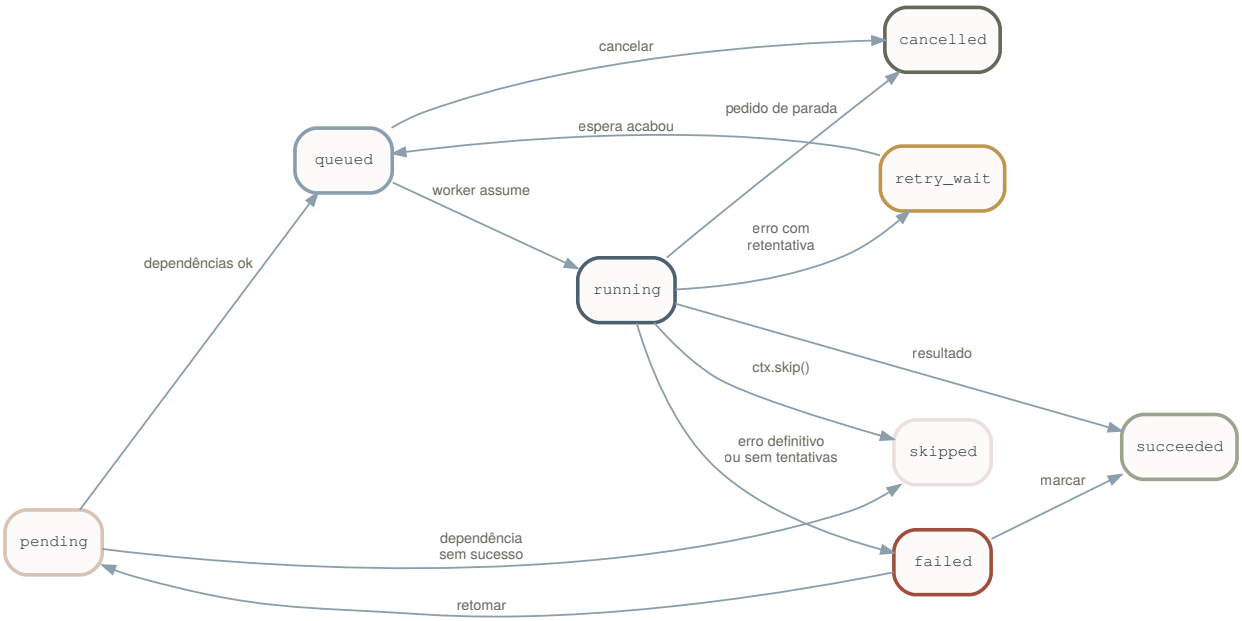
Arquitetura



Processos e fluxos de dados

Processo	Comando	Responsabilidade
API	`delco api`	Daphne com o Django: REST, WebSocket, painel, Swagger, métricas e migrações
Agendador	`delco scheduler`	Lê a pasta (e guarda o código), cria execuções agendadas, libera tarefas, encerra execuções que passaram do tempo limite, recupera leases vencidos, republica mensagens perdidas, envia notificações e manda sinal de vida ao Redis a cada 5 s
Worker	`delco worker`	Consome as filas, assume cada tarefa no banco e roda a tentativa num processo filho com heartbeat, tempo limite e cancelamento; obedece a pausas

Ciclo de vida de uma tarefa



Estados de task_instances.status

1. O agendador marca a tarefa como **na fila** no MySQL e só depois publica no Redis.
2. O worker trava a linha com `select_for_update()` e `status='queued'`; mensagem repetida não encontra mais a tarefa na fila.
3. A cada heartbeat o worker renova o lease. Se o servidor cair, o lease vence e a tarefa volta para nova tentativa.
4. O processo filho devolve resultado, erro (com a indicação se pode ser repetido) ou skip.
5. Tarefas terminadas guardam o resultado e não rodam de novo: só o passo interrompido é refeito.

Confiabilidade e retomada

- **MySQL é a fonte da verdade**; o Redis é só transporte.
- **Compare-and-set** em toda transição; `FOR UPDATE SKIP LOCKED` deixa vários agendadores trabalharem juntos.
- **Chave única por horário** impede duas execuções agendadas no mesmo horário.
- **Mensagens perdidas** são republicadas; o Redis pode ser apagado sem perda.
- **Worker pausado** devolve à fila mensagens que já tinha lido.
- **Tempo limite da execução**: o agendador confere a cada ciclo de recuperação, cancela o que falta, pede parada ao que roda e grava o motivo.

Sinais de vida

Workers gravam o heartbeat no MySQL. Agendadores gravam um sinal com validade no Redis (`delco:scheduler:<id>`), com ciclos, última leitura da pasta e último erro. A API usa esse sinal na saúde, no aviso do painel e na tela Sistema. Cada processo também escreve um arquivo de saúde usado pelo `delco healthcheck`.

Capítulo 6

Integrações

Tipo	Para que serve	O que configurar
Slack · Discord · Teams · Google Chat	Mensagem num canal	URL do webhook do canal
Telegram	Mensagem num grupo ou canal	Token do bot (BotFather) e ID do chat
E-mail	Mensagem para uma lista	Destinatários; servidor em `DELCO_SMTP_*`
Webhook genérico	Avisar outro sistema	URL; o corpo é JSON assinado com HMAC-SHA256
Gatilho por URL	Disparar um workflow de fora	Workflow e, se quiser, exigir assinatura

Eventos

Evento	Quando
`run.started`	A execução saiu da fila
`run.succeeded`	Terminou com sucesso
`run.failed`	Terminou com falha (inclusive tempo limite da execução)
`run.cancelled`	Foi cancelada
`run.recovered`	Terminou com sucesso logo depois de uma execução com falha do mesmo workflow

A entrega nasce na mesma transação que muda o estado da execução; o agendador envia e, se o destino recusar, tenta de novo até 5 vezes (10 s, 20 s, 40 s e 80 s). **Enviar teste** confere a configuração e entregas que desistiram podem ser **reenviadas** pelo histórico. As mensagens trazem as tarefas que falharam e o motivo.

Webhook genérico

```
POST https://seu-sistema/delco
X-Delco-Event: run.failed
X-Delco-Delivery: 42
X-Delco-Signature: sha256=<HMAC-SHA256 do corpo com o segredo>

{"event": "run.failed", "run": {"id": 118, "workflow": "sincronizar_catalogo", "status": "failed",
 "error": null, "failed_tasks": ["aplicar_lote_2"], "tasks_total": 7, "tasks_failed": 1, "url":
"…"}}
```

Gatilho por URL

A URL aparece uma única vez (só o SHA-256 do token fica no banco). Com **Exigir assinatura**, o Delco mostra também um segredo e recusa chamadas sem `X-Delco-Signature` ou `X-Hub-Signature-256` (o formato dos webhooks do GitHub). Cada gatilho aceita até `DELCO_HOOK_RATE_LIMIT` chamadas por minuto.

```
BODY='{"params": {"pasta_saida": "/tmp/rel"}, "note": "Build 42"}'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "$DELCO_HOOK_SECRET" | cut -d' ' -f2)
curl -X POST "$DELCO_HOOK_URL" -H 'Content-Type: application/json' \
  -H "X-Delco-Signature: sha256=$SIG" -d "$BODY"
```

E-mail

Configure `DELCO_SMTP_HOST`, `DELCO_SMTP_PORT`, `DELCO_SMTP_USER`, `DELCO_SMTP_PASSWORD`, `DELCO_SMTP_FROM` e `DELCO_SMTP_SECURITY` (`starttls`, `ssl` ou `none`). A mensagem sai em texto e HTML, com o link para a execução.

Capítulo 7

Linha de comando

Comandos de servidor e ferramentas

Comando	O que faz
`delco api` · `scheduler` · `worker`	Sobem os processos (também como `python manage.py ...`)
`delco migrate` · `seed` · `users create`	Banco, dados de demonstração e usuários
`delco new nome [--schedule]`	Cria um workflow a partir do modelo
`delco dev run nome [-p] [--json]`	Roda o workflow no terminal
`delco validate [pasta]`	Confere os arquivos
`delco doctor [--json] [--strict]`	Diagnóstico completo; código 1 se houver problema
`delco config [--json]`	Configuração em uso, segredos mascarados
`delco healthcheck api|scheduler|worker`	Usado pelo Docker

Comandos de cliente

Falam com a API usando o token salvo por `delco login` ou `DELCO_URL` e `DELCO_TOKEN`. Os de lista aceitam `--json`.

Comando	O que faz
`delco workflows` · `show nome` · `versions nome`	Lista, detalhe com parâmetros e histórico de versões
`delco trigger nome -p k=v [--wait] [--dry-run]`	Dispara (ou só valida os parâmetros)
`delco backfill nome --from D --to D [--dry-run] [--all]`	Reprocessa um período
`delco runs [--status] [--trigger] [--since] [-q]`	Lista execuções
`delco run id` · `logs id tarefa [-f] [--level]`	Detalhe e log (acompanhando ao vivo)
`delco cancel` · `retry [--from]` · `mark --status` · `note`	Ações sobre execuções e tarefas
`delco pause` · `resume`	Agenda de um workflow
`delco workers [pause|resume id]`	Lista, pausa e retoma workers
`delco upcoming` · `integrations` · `export arquivo.csv`	Próximas execuções, integrações e CSV

Capítulo 8

API

Todas as rotas ficam em `/api`, respondem JSON e exigem `Authorization: Bearer <token>`, exceto login, saúde e o gatilho por URL. A documentação interativa fica em `/api/docs` e o esquema em `/api/openapi.json`.

Grupo	Rotas	Papel
Acesso	`POST /auth/login` · `/auth/logout` · `GET /auth/me` · `PUT /auth/me/password` · `GET` / `PATCH /auth/me/preferences` · `/tokens`	todos
Workflows	`GET /workflows` · `/workflows/{n}` · `/grid` · `/versions` · `/source`	leitura
	`POST /workflows/{n}/runs` (com `dry_run`) · `/backfill` · `/pause` · `/resume` · `POST /workflows/bulk`	operador
Execuções	`GET /runs` · `/runs/export` · `/runs/{id}` · `/runs/{id}/tasks/{t}` · `/attempts/{id}/logs`	leitura
	`PATCH /runs/{id}` · `POST /cancel` · `/retry` · `/tasks/{t}/mark` · `POST /runs/bulk`	operador
	`DELETE /runs/{id}` · `bulk` com `delete`	admin
Monitoramento	`/overview` · `/insights` · `/upcoming` · `/workers` · `/queues` · `/health` · `/metrics` · `WS /ws`	leitura
	`POST /workers/{id}/pause` · `/resume`	operador
Integrações	`/integrations` · `/{id}` · `/test` · `/deliveries` · `/deliveries/{d}/retry` · `POST /hooks/{token}`	operador / admin
Administração	`/users` · `/audit` · `/audit/export` · `GET /system` · `POST /system/housekeeping`	admin

Erros

```
{"code": "validation_error", "message": "Confira os parâmetros destacados.",
 "fields": {"params.dias": "Precisa ser no máximo 30."}}
```

Status	Código	Quando
401	`unauthenticated` · `invalid_token` · `invalid_signature`	Sem token, token inválido ou gatilho sem assinatura
403	`forbidden`	Papel sem permissão
404	`not_found`	Recurso não existe
409	`conflict`	Ação impossível no estado atual
422	`validation_error`	Campos inválidos
423	`locked`	Conta bloqueada por erros de senha
429	`rate_limited`	Limite de chamadas por minuto

Exemplos

```
curl -s -X POST localhost:8080/api/workflows/relatorio_vendas_diario/backfill \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"start": "2026-10-01", "end": "2026-10-07", "dry_run": true}'

curl -s -X POST localhost:8080/api/runs/118/tasks/extrair_pedidos/mark \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' -d '{"status":
"succeeded"}'
```

Capítulo 9

Configuração

Tudo vem das variáveis `DELCO_*` (modelo em `.env.example`). Ao subir, cada valor é conferido; se houver problemas, o processo para e lista todos de uma vez, como “DELCO_LEASE_SECONDS precisa ser pelo menos o dobro de DELCO_HEARTBEAT_SECONDS”.

Variável	Padrão	Para que serve
`DELCO_ENVIRONMENT`	`development`	`production` liga os avisos de produção
`DELCO_DATABASE_URL`	local	`mysql://usuario:senha@host:3306/banco`
`DELCO_REDIS_URL`	local	`redis://host:6379/0`
`DELCO_SECRET_KEY`	sorteada	Chave do Django
`DELCO_ALLOWED_HOSTS` · `DELCO_PUBLIC_URL`	`*` · localhost	Domínio do painel e links das notificações
`DELCO_SEED`	`true`	Demonstração num banco vazio
`DELCO_WORKFLOWS_DIR` · `DELCO_TIMEZONE`	`workflows/` · São Paulo	Pasta e fuso padrão
`DELCO_WORKER_QUEUES` · `DELCO_WORKER_CONCURRENCY`	`default` · `4`	Filas e vagas por worker
`DELCO_HEARTBEAT_SECONDS` · `DELCO_LEASE_SECONDS`	`5` · `30`	Sinal e posse das tarefas
`DELCO_REDELIVER_AFTER_SECONDS` · `DELCO_WORKER_GRACE_SECONDS`	`60` · `30`	Republicação e desligamento gracioso
`DELCO_TICK_SECONDS` · `DELCO_SYNC_SECONDS` · `DELCO_RECOVERY_SECONDS`	`1` · `30` · `5`	Ritmo do agendador
`DELCO_RETENTION_DAYS`	`30`	Histórico guardado
`DELCO_MAX_OUTPUT_BYTES` · `DELCO_MAX_LOG_LINES`	1 MiB · `5000`	Limites por tentativa
`DELCO_SESSION_HOURS` · `DELCO_LOGIN_MAX_FAILURES` · `DELCO_LOGIN_LOCK_MINUTES`	`12` · `5` · `5`	Sessão e bloqueio
`DELCO_LOGIN_RATE_LIMIT` · `DELCO_HOOK_RATE_LIMIT`	`30` · `60`	Chamadas por minuto (0 desliga)
`DELCO_METRICS_TOKEN`	vazio	Token aceito em `/api/metrics`
`DELCO_MAX_BACKFILL_RUNS`	`366`	Limite de um reprocessamento
`DELCO_WEBHOOK_TIMEOUT`	`10`	Espera pelos destinos
`DELCO_SMTP_*`	vazio	Servidor de e-mail
`DELCO_TELEGRAM_API_URL`	API oficial	Para proxies
`DELCO_LOG_LEVEL` · `DELCO_LOG_FORMAT`	`INFO` · `text`	Logs em texto ou JSON

Variável	Padrão	Para que serve
`DELCO_HEALTH_DIR`	pasta temporária	Arquivos do healthcheck

Capítulo 10

Operação em produção

Checklist

- `DELCO_ENVIRONMENT=production`, `DELCO_SEED=false`, `DELCO_SECRET_KEY` longa, `DELCO_ALLOWED_HOSTS` e `DELCO_PUBLIC_URL` com o domínio real.
- HTTPS num proxy reverso (Nginx, Caddy, Traefik) repassando `X-Forwarded-Proto` e o WebSocket em `/api/ws`.
- Contas de demonstração desativadas ou com outra senha; `delco doctor --strict` sem avisos.
- Backup diário do MySQL (o Redis pode ser perdido sem perda de dados).

Diagnóstico

```
docker compose exec api delco doctor
CONFIGURAÇÃO
  ✓ Chave secreta: Definida em DELCO_SECRET_KEY.
PROCESSOS
  ✓ Agendador: 1 ativo(s): api-1
  ! Filas sem worker: Nenhum worker online consome: relatorios
      → Inclua a fila em DELCO_WORKER_QUEUES de algum worker.
```

Métricas

```
scrape_configs:
  - job_name: delco
    metrics_path: /api/metrics
    authorization:
      credentials: <DELCO_METRICS_TOKEN>
    static_configs:
      - targets: ["delco.suaempresa.com"]
```

Séries: `delco_runs_active`, `delco_runs_finished_24h`, `delco_workflow_last_success_timestamp_seconds`, `delco_tasks`, `delco_workers`, `delco_worker_slots`, `delco_worker_slots_busy`, `delco_schedulers`, `delco_deliveries`, `delco_workflow_paused` e `delco_workflow_load_errors`. Um bom alerta: `delco_schedulers == 0` por 2 minutos.

Healthchecks, escala e manutenção

- O `docker-compose.yml` usa `delco healthcheck` na API, no agendador e nos workers.
- Escale workers com `--scale worker=N` e separe filas pesadas com `DELCO_WORKER_QUEUES`.
- Antes de mexer numa máquina, **pause o worker** pelo painel ou com `delco workers pause id`.
- Atualizações: `git pull` e `docker compose up -d --build`; a API aplica as migrações com trava.
- Logs em JSON com `DELCO_LOG_FORMAT=json`, prontos para Loki, Elastic ou Datadog.

Capítulo 11

Segurança

Regra	Como funciona
Senhas	scrypt do Django, mínimo de 8 caracteres com letras e números, bloqueio depois de erros seguidos
Tokens	Opacos; só o SHA-256 fica no banco; validade e revogação
Limite de chamadas	Login por IP e gatilho por token, com contagem no Redis
Papéis	Leitura, Operador e Administrador em cada rota
Integrações	URLs e token do bot ocultos; webhook assinado; gatilho com assinatura opcional
Parâmetros secretos	`Param(secret=True)` mascara o valor no painel, na API e na auditoria
Isolamento	Cada tentativa num processo filho com grupo próprio
Auditoria	Toda ação que muda algo, com filtros e CSV, por 180 dias
Cabeçalhos	CSP, `X-Frame-Options`, `nosniff`, `Cache-Control: no-store` na API
Verificações	Tela Sistema e `delco doctor` apontam contas de demonstração, chave e domínios de produção

Capítulo 12

Testes e qualidade

São 103 testes com MySQL e Redis reais e cobertura acima de 90%. Eles sobem workers e agendadores de verdade e cobrem a retomada no passo 23 de 50, mensagens perdidas e repetidas, Redis apagado, agendadores concorrentes, parâmetros tipados, `skip`, `Fail`, `retry_on`, tempo limite da execução, reprocessamento, marcação manual, ações em lote, versões, worker pausado, métricas, `delco doctor`, preferências, auditoria, limites de chamadas, e-mail por SMTP, Telegram, “voltou a funcionar” e gatilho assinado.

```
python -m pytest              # suíte completa
ruff check .                  # lint
make lint                     # Ruff e sintaxe do painel
```

O CI roda Ruff, a sintaxe do JavaScript, os workflows de exemplo com `delco dev run`, a suíte com MySQL 8.4 e Redis, e monta e testa a imagem Docker. O Dependabot propõe atualizações semanais.

Capítulo 13

Solução de problemas

Sintoma	Causa provável e solução
Aviso “Nenhum agendador ativo”	O processo do agendador parou: `docker compose up -d scheduler` e veja `docker compose logs scheduler`
Tarefa parada em “Na fila”	Nenhum worker consome aquela fila ou todos estão pausados: veja a tela Sistema (“Filas sem worker”)
Processo não sobe e lista problemas	Alguma variável `DELCO_*` inválida; corrija conforme a mensagem
422 com `params.x`	Parâmetro fora do tipo, das opções ou dos limites do `Param`
Execução falhou com “Tempo limite da execução”	`run_timeout` do workflow; aumente ou divida o trabalho
Tarefa falhou sem retentativas	Erro fora de `retry_on` ou `Fail`; o log diz “Erro marcado como definitivo”
Workflow some da lista	Erro de importação no arquivo: veja o aviso em Workflows ou `delco validate`
Notificação não chega	Histórico de entregas da integração; use **Enviar teste** e **Reenviar**
E-mail não pode ser criado	Falta `DELCO_SMTP_HOST`
429 no login ou no gatilho	Limite por minuto; ajuste `DELCO_LOGIN_RATE_LIMIT` ou `DELCO_HOOK_RATE_LIMIT`
401 no gatilho por URL	Gatilho com assinatura: envie `X-Delco-Signature`
Painel sem atualizar sozinho	O indicador “Ao vivo” mostra a conexão; o proxy precisa repassar WebSocket em `/api/ws`